คู่มือปฏิบัติการ · รายวิชา 5534408 การพัฒนาแอปพลิเคชันบนอุปกรณ์เคลื่อนที่

KitchenAI

สร้างแอป Flutter ที่คุยกับ AI (Gemini): แนะนำเมนูจากวัตถุดิบ และดูรูปแล้วบอกว่ามีอะไร

ฉบับนักศึกษา · เรียนรู้ด้วยตนเอง · Step 1–9 · ใช้ของฟรีทั้งหมด (Flutter · Gemini API free tier · GitHub)

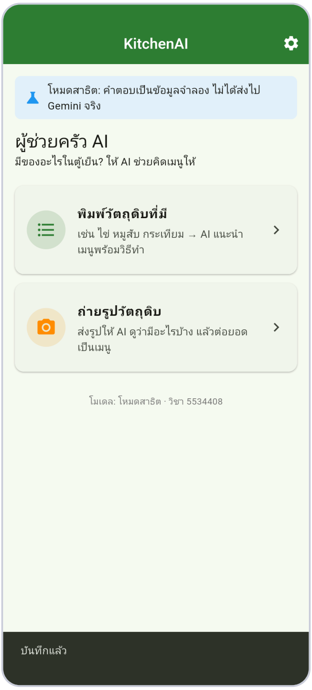
ใช้ Android Studio · ต่อยอดจากความรู้ http / JSON / shared_preferences / image_picker ที่เรียนจาก LabStock

โค้ด: https://github.com/kittipongnuanyai/KitchenAI · tag starter (เริ่มทำ) · done (เฉลย) · ปรับปรุง 15 ต.ค. 2569

ก่อนเริ่ม: แอปนี้ทำอะไร และคุณจะได้เรียนอะไร

KitchenAI เป็นแอปมือถือเล็ก ๆ ที่ส่งคำถามไปให้ AI (Google Gemini) แล้วเอาคำตอบมาแสดงเป็นหน้าจอสวย ๆ มี 2 ฟีเจอร์:

ฟีเจอร์	ผู้ใช้ทำอะไร	AI ทำอะไร	เทคนิค
เมนูจากวัตถุดิบ	พิมพ์ของที่มี เช่น ไข่ หมูสับ กระเทียม	คิดเมนู 3 อย่าง บอกวัตถุดิบ ของที่ต้องซื้อเพิ่ม วิธีทำทีละขั้น	HTTP POST · JSON · prompt · บังคับ AI ตอบเป็น JSON
ถ่ายรูปวัตถุดิบ	ถ่ายรูปของในตู้เย็น	บอกว่าเห็นอะไร ปริมาณเท่าไร แล้วส่งต่อไปคิดเมนู	image_picker · base64 · ส่งรูป+ข้อความพร้อมกัน (multimodal)



โหมดสาธิต: คำตอบเป็นข้อมูลจำลอง ไม่ได้ส่งไป Gemini จริง

ผู้ช่วยครัว AI

มีของอะไรในตู้เย็น? ให้ AI ช่วยคิดเมนูให้

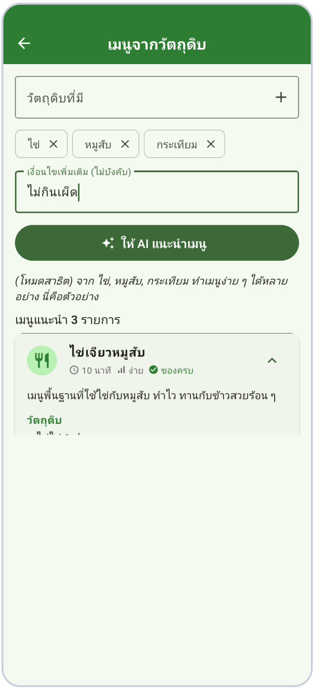
พิมพ์วัตถุดิบที่มี
เช่น ไข่ หมูสับ กระเทียม → AI แนะนำเมนูพร้อมวิธีทำ

ถ่ายรูปวัตถุดิบ
ส่งรูปให้ AI ดูว่ามีอะไรบ้าง แล้วต่อยอดเป็นเมนู

โมเดล: โหมดสาธิต · วิชา 5534408

บันทึกแล้ว

หน้าแรก



เงื่อนไขเพิ่มเติม (ไม่บังคับ): ไม่กินเผ็ด

(โหมดสาธิต) จาก ไข่, หมูสับ, กระเทียม ทำเมนูง่าย ๆ ได้หลายอย่าง นี่คือตัวอย่าง

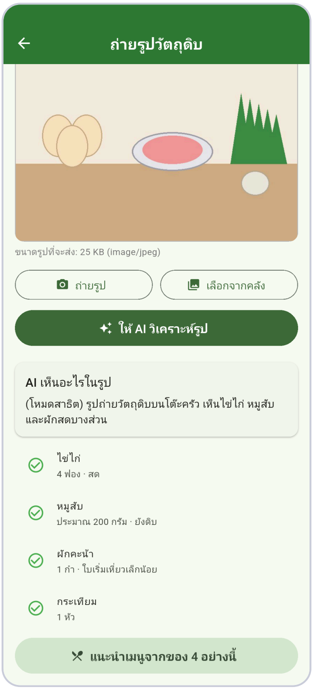
เมนูแนะนำ 3 รายการ

ไข่เจียวหมูสับ
10 นาที · ง่าย · ของครบ

เมนูพื้นฐานที่ใช้ไข่กับหมูสับ ทำไว ทานกับข้าวสวยร้อน ๆ

วัตถุดิบ

เมนูจากวัตถุดิบ



ขนาดรูปที่จะส่ง: 25 KB (image/jpeg)

AI เห็นอะไรในรูป

(โหมดสาธิต) รูปถ่ายวัตถุดิบบนโต๊ะครัว เห็นไข่ไก่ หมูสับ และผักสดบางส่วน

- ✓ ไข่ไก่ — 4 ฟอง · สด
- ✓ หมูสับ — ประมาณ 200 กรัม · ยังดิบ
- ✓ ผักคะน้า — 1 กำ · ใบเริ่มเหี่ยวเล็กน้อย
- ✓ กระเทียม — 1 หัว

ผลวิเคราะห์รูป

คำศัพท์ที่จะเจอตลอดเล่ม

คำ	ความหมายแบบบ้าน ๆ
LLM / โมเดล	โปรแกรม AI ที่รับข้อความ (และรูป) แล้วเขียนข้อความตอบ เช่น Gemini, GPT, Claude — เราไม่ได้รันมันเอง แต่เรียกผ่านอินเทอร์เน็ต
API	ช่องทางให้โปรแกรมคุยกับโปรแกรม — ของ Gemini คือ URL ที่รับ JSON แล้วตอบ JSON เหมือน PHP API ใน LabStock
API key	รหัสประจำตัวที่แนบไปทุกคำขอ บอกว่าเป็นเรา และนับโควตาของเรา
prompt	ข้อความที่เราส่งไปถาม/สั่ง AI · **system instruction** = prompt พิเศษที่บอกบทบาทและกติกา ใช้กับทุกคำถาม
token	หน่วยนับความยาวข้อความที่ AI ใช้ (คำไทย 1 คำ ≈ 1–3 token) โควตาฟรีนับเป็น token และคำขอต่อนาที/วัน

temperature	ตัวเลข 0–2 ควบคุมความสุ่ม: ต่ำ = ตอบเหมือนเดิมทุกครั้ง · สูง = หลากหลาย/สร้างสรรค์
JSON schema	แบบฟอร์มบอกโครงสร้าง JSON ที่ต้องการ ส่งไปให้ AI กรอกให้ตรง จะได้แปลงเป็น object ใน Dart ได้
multimodal	ส่งได้มากกว่าข้อความ — แล็บนี้ส่งรูปภาพไปพร้อมคำถาม
base64	วิธีเขียนไฟล์ไบนารี (รูป) เป็นตัวอักษร เพื่อยัดลงใน JSON ได้

สิ่งที่ต้องมี

- Android Studio + Flutter 3.35 ขึ้นไป (เครื่องที่ใช้ทำ LabStock ใช้ได้เลย) ตรวจด้วย `flutter doctor`
- บัญชี Google สำหรับขอ API key (Step 1) และอินเทอร์เน็ตตอนทดสอบกับ AI จริง
- อีมูเลเตอร์หรือมือถือ Android (มือถือจริงจะได้ลองถ่ายรูปจริง)
- **ไม่ต้องมี** XAMPP, บัตรเครดิต หรือเซิร์ฟเวอร์ใด ๆ — ทุกอย่างในแล็บนี้ฟรี

ลำดับที่จะทำ

Step	ประเภท	หัวข้อ	ทำเสร็จแล้วจะได้อะไร	นาที
1	อ่าน+ทำตาม	**ขอ Gemini API key (ฟรี) และทดลองยิงคำขอแรก**	มี API key ของตัวเอง และเห็นคำตอบจาก Gemini บนหน้าจอด้วย curl ก่อนเขียนแอป	25
2	อ่าน+ทำตาม	**โหลดโค้ดเริ่มต้น เปิดใน Android Studio และรันครั้งแรก**	แอปรันบนอีมูเลเตอร์ได้ เห็นทุกหน้าในโหมดสาธิต และรู้ว่าเทสต์ไหนยัง fail (คือสิ่งที่ต้องทำ)	25
3	อ่าน+ทำตาม	**ทัวร์โค้ดที่ให้มา: ข้อมูลไหลจากหน้าจอไปถึง Gemini อย่างไร**	วาดภาพในหัวได้ว่าแต่ละไฟล์ทำหน้าที่อะไร และจุดที่ต้องเขียนเองอยู่ตรงไหนของเส้นทาง	30
4	อ่าน+ทำตาม	**เข้าใจ Gemini API: JSON ที่ส่งไปและ JSON ที่ได้กลับมา**	อ่าน/เขียน request และ response ของ generateContent ได้ทุกช่อง ก่อนลงมือเขียนโค้ด	25
5	ทำเอง	**เขียน GeminiClient: ส่งคำขอและแกะคำตอบ**	generate() ยิง HTTP ไป Gemini ได้จริง · extractText() แกะข้อความได้ · เทสต์ 6 ข้อผ่าน · ปุ่ม "ทดสอบการเชื่อมต่อ" ได้คำตอบจาก Gemini จริง	50
6	ทำเอง	**หน้า "เมนูจากวัตถุดิบ": prompt, JSON schema และการแสดงผล**	พิมพ์วัตถุดิบ → กดปุ่ม → ได้การ์ดเมนู 3 รายการจาก Gemini จริง พร้อมวัตถุดิบ ของที่ต้องซื้อ และวิธีทำ	45
7	ทำเอง	**หน้า "ถ่ายรูปวัตถุดิบ": ส่งรูปให้ AI ดู**	ถ่าย/เลือกรูป → AI บอกว่าเห็นอะไร → กดปุ่มส่งต่อไปขอเมนูจากของในรูป	45
8	อ่าน+ทำตาม	**ตรวจสอบรอบสุดท้าย แก้ปัญหา และสร้าง APK**	เทสต์ผ่านครบ · ไม่มีคำเตือนจาก analyzer · ได้ไฟล์ APK ติดตั้งบนมือถือ	20
9	ทำเอง	**โจทย์ท้าทาย (เลือกทำ) และสิ่งที่ต้องส่ง**	ต่อยอดแอปด้วยตัวเองอย่างน้อย 1 ข้อ และส่งงานครบ	60
		รวม		**325**

ลิงก์

ต้องการ	ลิงก์ / คำสั่ง
ขอ API key	https://aistudio.google.com/apikey
โค้ดเริ่มต้น (ZIP)	https://github.com/kittipongnuanyai/KitchenAI/archive/refs/tags/starter.zip
ใน Android Studio	Get from VCS → `https://github.com/kittipongnuanyai/KitchenAI.git` → Git → Branches → Tags → `starter` → Checkout
ดูเฉลยเทียบกับ starter	https://github.com/kittipongnuanyai/KitchenAI/compare/starter...done
เอกสาร Gemini API	https://ai.google.dev/gemini-api/docs · รายชื่อโมเดล: /models · โควตา: https://aistudio.google.com/rate-limit

กติกา

- Step 5–7 ต้องเขียนเอง ห้ามคัดลอกเฉลยทั้งก้อน — เฉลยเปิดให้ดู**หลัง**พยายามเองแล้ว และผู้สอนจะสุ่มถามให้อธิบายโค้ดบรรทัดต่อบรรทัด
- ติดเกิน 30 นาที ถามในกลุ่มพร้อมแนบ error และบอกว่าลองอะไรไปแล้ว
- **ห้ามให้ API key หลุดขึ้น GitHub หรือกลุ่มแชต** ไม่ว่ากรณีใด

STEP 1 · อ่าน+ทำตาม

ขอ Gemini API key (ฟรี) และทดลองยิงคำขอแรก

เวลาโดยประมาณ 25 นาที

เป้าหมาย: มี API key ของตัวเอง และเห็นคำตอบจาก Gemini บนหน้าจอด้วย curl ก่อนเขียนแอป

ก่อนจะให้แอป "คุยกับ AI" ได้ เราต้องมี **API key** ซึ่งเป็นรหัสผ่านยาว ๆ ที่บอก Google ว่าคำขอนี้มาจากบัญชีของเรา Google ให้ใช้ Gemini ฟรีในปริมาณจำกัดต่อวัน (เพียงพอสำหรับทำแล็บนี้ทั้งเทอม) ไม่ต้องใส่บัตรเครดิต

1.1 สร้าง key ใน Google AI Studio

1. เปิดเบราว์เซอร์ไปที่ **https://aistudio.google.com/apikey** แล้ว sign in ด้วยบัญชี Google (ใช้ Gmail ส่วนตัวได้ บัญชีมหาวิทยาลัยบางแห่งอาจถูกแอดมินปิดไว้ ถ้ากดสร้างไม่ได้ให้สลับบัญชี)
2. ครั้งแรกจะมีหน้าให้ยอมรับข้อตกลง (Terms of Service) กดยอมรับ
3. กดปุ่ม **Create API key** → ถ้าถามให้เลือกโปรเจกต์ ให้เลือก *Create API key in new project* หรือโปรเจกต์ที่ระบบสร้างให้ (ชื่อ Gemini API / Default)
4. ระบบจะแสดง key ขึ้นต้นด้วย `AIza` ยาวประมาณ 39 ตัวอักษร กด **Copy** แล้ววางเก็บไว้ในที่ปลอดภัย เช่น ไฟล์ Notes ส่วนตัว (ยังไม่ต้องวางลงในโค้ด)

กติกาเรื่อง key — สำคัญมาก

- key = เงินของคุณ ใครได้ key ไปก็ใช้โควตาแทนคุณได้ **ห้ามโพสต์ในกลุ่ม ห้ามใส่ในสไลด์ ห้าม commit ขึ้น GitHub**
- แอปนี้ออกแบบให้ใส่ key ผ่านหน้า "ตั้งค่า" และเก็บไว้ในเครื่องเท่านั้น จึงไม่มี key อยู่ในโค้ดเลย
- ถ้าเผลอหลุด ให้กลับไปที่ aistudio.google.com/apikey กด **Delete** key เดิมแล้วสร้างใหม่ ใช้เวลาไม่ถึงนาที
- Google จำกัดโควตาฟรี "ต่อนาที" และ "ต่อวัน" (ดูตัวเลขปัจจุบันที่ aistudio.google.com/rate-limit) ถ้ากดถี่ ๆ จะได้ error 429 — รอสักครู่แล้วลองใหม่

1.2 ยิงคำขอแรกด้วย curl (ยังไม่ต้องใช้ Flutter)

เราจะคุยกับ Gemini ด้วยคำสั่ง `curl` ก่อน เพื่อให้เห็นว่า "API" ก็คือการส่ง JSON ไปที่ URL หนึ่ง แล้วได้ JSON กลับมา — แอป Flutter ของเราก็จะทำแบบเดียวกันเป๊ะ (Windows 10/11 มี curl ติดมาแล้ว เปิด **PowerShell** หรือ Terminal ใน Android Studio ก็ได้ · macOS เปิด Terminal)

macOS / Linux (แทน `ใส่KEYของคุณ` ด้วย key จริง):

```
curl "https://generativelanguage.googleapis.com/v1beta/models/gemini-3.8-flash:generateCon
tent" \
  -H "x-goog-api-key: ใส่KEYของคุณ" \
  -H "Content-Type: application/json" \
  -d '{"contents":[{"parts":[{"text":"มีไข่กับหมูสับ ทำอะไรกินได้บ้าง ตอบสั้นๆ"}]}]}'
```

Windows PowerShell (ต้องใช้ `curl.exe` เพราะ `curl` เฉย ๆ ใน PowerShell เป็นคำสั่งอื่น และต้องใส่ body ผ่านไฟล์เพื่อเลี่ยงปัญหาเครื่องหมายคำพูด):

```
# 1) สร้างไฟล์ body.json ด้วย Notepad ให้มีข้อความนี้ (บันทึกเป็น UTF-8)
{"contents":[{"parts":[{"text":"มีไข่กับหมูสับ ทำอะไรกินได้บ้าง ตอบสั้นๆ"}]}]}

# 2) รันใน PowerShell ที่โฟลเดอร์เดียวกับ body.json
curl.exe "https://generativelanguage.googleapis.com/v1beta/models/gemini-3.8-flash:generateContent" -H "x-goog-api-key: ใส่KEYของคุณ" -H "Content-Type: application/json; charset=utf-8" --data-binary "@body.json"
```

ถ้าสำเร็จจะได้ JSON ก้อนใหญ่ หน้าตาประมาณนี้ (ตัดให้สั้น):

```
{
  "candidates": [
    {
      "content": {
        "parts": [ { "text": "ทำได้หลายเมนูเลยครับ เช่น\n1. ไข่เจียวหมูสับ ..." } ],
        "role": "model"
      },
      "finishReason": "STOP"
    }
  ],
  "usageMetadata": { "promptTokenCount": 21, "candidatesTokenCount": 180, "totalTokenCount": 201 },
  "modelVersion": "gemini-3.8-flash"
}
```

อ่าน JSON นี้ให้ออก — ทั้งแอปใช้แค่นี้

- `candidates[0].content.parts[0].text` = ข้อความคำตอบ (คำตอบอาจแบ่งเป็นหลาย part ต้องเอามาต่อกัน)
- `finishReason` = ทำไมถึงหยุดตอบ: `STOP` ปกติ · `MAX_TOKENS` ยาวเกินที่กำหนด · `SAFETY` ถูกบล็อก
- `usageMetadata` = ใช้ไปกี่ token (หน่วยนับความยาวข้อความ ประมาณ 1 token ≈ 1–3 ตัวอักษรไทย) — ตัวเลขนี้คือสิ่งที่โควตานับ
- ถ้าได้ `{"error": {"code": 400, "message": "API key not valid..."}}` แปลว่า key ผิด/คัดลอกไม่ครบ · `429` = ยิงถี่เกินโควตา

✔ Checkpoint 1

- มี API key เก็บไว้แล้ว และรู้ว่าจะไปลบ/สร้างใหม่ที่ไหน
- curl ได้คำตอบภาษาไทยจาก Gemini (ถ่ายภาพหน้าจอผลลัพธ์ไว้ส่งงาน)
- ชี้ได้ว่าข้อความคำตอบอยู่ตรงไหนของ JSON

STEP 2 · อ่าน+ทำตาม

โหลดโค้ดเริ่มต้น เปิดใน Android Studio และรันครั้งแรก

เวลาโดยประมาณ 25 นาที

เป้าหมาย: แอปรันบนอีมูเลเตอร์ได้ เห็นทุกหน้าในโหมดสาธิต และรู้ว่าเทสต์ไหนยัง fail (คือสิ่งที่ต้องทำ)

โค้ดเริ่มต้น (tag `starter`) มี UI ทุกหน้า โมเดลข้อมูล การตั้งค่า โหมดสาธิต และเทสต์ครบแล้ว **เว้นไว้เฉพาะส่วน "คุยกับ AI"** ใน 4 ไฟล์ ซึ่งคุณจะเขียนเองใน Step 5–7 (ค้นหาคำว่า `TODO(Step` ในโปรเจกต์จะเจอทุกจุด)

2.1 โหลดโค้ด

วิธี	ทำอย่างไร

git (แนะนำ)	`git clone https://github.com/kittipongnuanyai/KitchenAI.git kitchen_ai`
	`cd kitchen_ai`
	`git checkout starter`
	`flutter pub get`

ใน Android Studio	File → New → Project from Version Control → วาง URL `https://github.com/kittipongnuanyai/KitchenAI.git` → Clone แล้วไปที่ Git → Branches... → Tags → starter → Checkout
ZIP (ไม่ใช้ git)	ดาวน์โหลด `https://github.com/kittipongnuanyai/KitchenAI/archive/refs/tags/starter.zip` แตกไฟล์ แล้ว File → Open โฟลเดอร์ที่แตกออกมา

เปิดโปรเจกต์แล้วรอให้ Android Studio ทำ `flutter pub get` จนเสร็จ (มุมล่างขวามีแถบความคืบหน้า) ถ้ามันไม่ทำเอง เปิดไฟล์ `pubspec.yaml` แล้วกด Pub get ที่แถบด้านบน

2.2 แพ็กเกจที่ใช้

`pubspec.yaml` เฉพาะส่วน dependencies

```yaml
dependencies:
  cupertino_icons: ^2.0.0
  flutter:
    sdk: flutter
  http: ^1.6.0
  image_picker: ^1.2.4
  shared_preferences: ^2.5.6
```

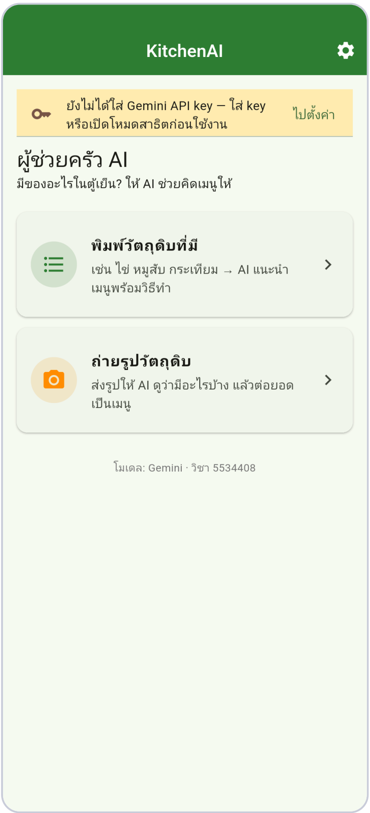
แพ็กเกจ	ใช้ทำอะไร	เคยใช้ใน LabStock?
`http`	ยิง HTTP POST ไป Gemini	ใช่ (เรียก PHP API)
`image_picker`	ถ่ายรูป / เลือกรูปจากคลัง	ใช่ (อัปโหลดรูปอุปกรณ์)

`shared_preferences`	เก็บ API key และโหมดสาธิตไว้ในเครื่อง	ใช่ (จำการเข้าสู่ระบบ)

สังเกตว่า**ไม่มีแพ็กเกจ AI พิเศษ**เลย — เราคุยกับ Gemini ด้วย HTTP + JSON ล้วน ๆ เหมือนที่คุยกับ PHP ใน LabStock ต่างกันแค่ URL และรูปแบบ JSON เท่านั้น นี่คือจุดประสงค์ของแล็บนี้: ให้เห็นว่า "AI API" ไม่ได้มีเวทมนตร์อะไร

2.3 รันครั้งแรก

1. เปิด Device Manager สร้าง/เปิดอีมูเลเตอร์ (Pixel อะไรก็ได้ API 34 ขึ้นไป) หรือเสียบมือถือจริงที่เปิด USB debugging
2. กด **Run** ▶ (หรือ `flutter run`) รอ build ครั้งแรก 2–5 นาที
3. หน้าแรกจะมีแถบเหลืองเตือนว่า "ยังไม่ได้ใส่ Gemini API key" กด **ไปตั้งค่า**
4. เปิดสวิตช์ **โหมดสาธิต** → กด **ทดสอบการเชื่อมต่อ** (ได้ข้อความจำลอง) → **บันทึก**
5. ลองกด "พิมพ์วัตถุดิบที่มี" พิมพ์ ไข่ แล้วกด + → กด "ให้ AI แนะนำเมนู" — ใน starter จะขึ้น SnackBar ว่า *ยังไม่ได้เขียน _ask()* นั่นถูกต้องแล้ว เพราะคุณยังไม่ได้ทำ Step 6


KitchenAI

ยังไม่ได้ใส่ Gemini API key — ใส่ key หรือเปิดโหมดสาธิตก่อนใช้งาน

ผู้ช่วยครัว AI

มีของอะไรในตู้เย็น? ให้ AI ช่วยคิดเมนูให้

พิมพ์วัตถุดิบที่มี

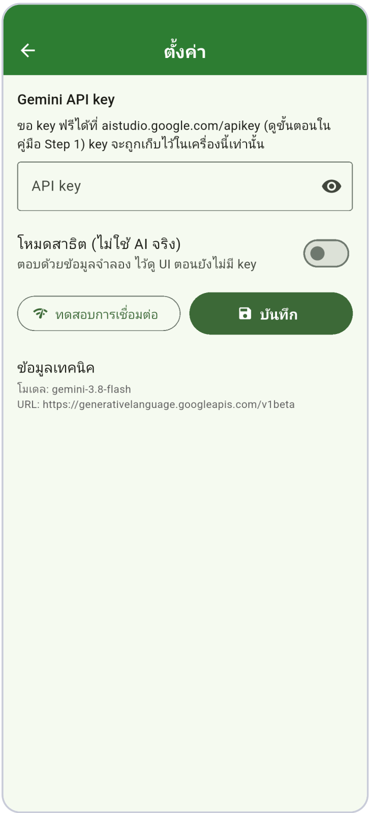
เช่น ไข่ หมูสับ กระเทียม → AI แนะนำเมนูพร้อมวิธีทำ

ถ่ายรูปวัตถุดิบ

ส่งรูปให้ AI ดูว่ามีอะไรบ้าง แล้วต่อยอดเป็นเมนู

โมเดล: Gemini · วิชา 5534408

หน้าแรกตอนยังไม่มี key


ตั้งค่า

Gemini API key

ขอ key ฟรีได้ที่ aistudio.google.com/apikey (ดูขั้นตอนในคู่มือ Step 1) key จะถูกเก็บไว้ในเครื่องนี้เท่านั้น

API key

โหมดสาธิต (ไม่ใช้ AI จริง) ○

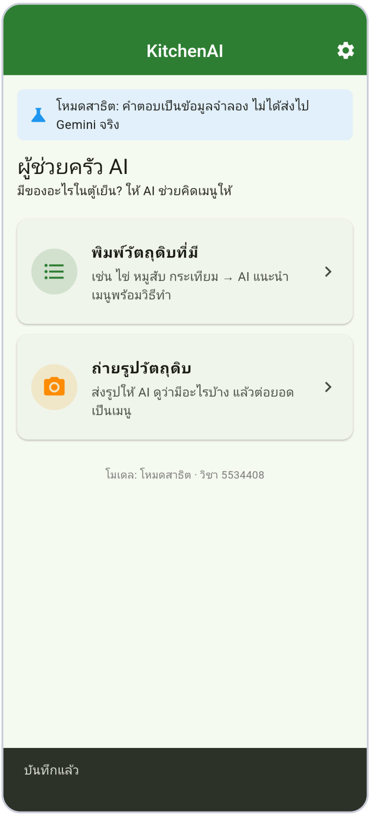
ตอบด้วยข้อมูลจำลอง ไว้ดู UI ตอนยังไม่มี key

ข้อมูลเทคนิค

โมเดล: gemini-3.8-flash

URL: https://generativelanguage.googleapis.com/v1beta

หน้าตั้งค่า


KitchenAI

โหมดสาธิต: คำตอบเป็นข้อมูลจำลอง ไม่ได้ส่งไป Gemini จริง

ผู้ช่วยครัว AI

มีของอะไรในตู้เย็น? ให้ AI ช่วยคิดเมนูให้

พิมพ์วัตถุดิบที่มี

เช่น ไข่ หมูสับ กระเทียม → AI แนะนำเมนูพร้อมวิธีทำ

ถ่ายรูปวัตถุดิบ

ส่งรูปให้ AI ดูว่ามีอะไรบ้าง แล้วต่อยอดเป็นเมนู

โมเดล: โหมดสาธิต · วิชา 5534408

บันทึกแล้ว

เปิดโหมดสาธิตแล้ว

2.4 รันเทสต์ดูว่าอะไร "ยังไม่เสร็จ"

โปรเจกต์มีเทสต์อัตโนมัติ 12 ข้อ ใน starter จะ **fail 7 ข้อ** ซึ่งก็คือ 7 สิ่งที่คุณต้องทำให้ผ่าน ใช้มันเป็นเช็กลิสต์ได้เลย:

```
flutter test
```

ไฟล์เทสต์	ใน starter	จะผ่านเมื่อทำ

`test/gemini_client_test.dart` 6 ข้อ	fail 5 / ผ่าน 1	Step 5 (ข้อ "ไม่มี key" ผ่านตั้งแต่แรกเพราะเช็กไว้ให้แล้ว)
`test/widget_test.dart` 6 ข้อ	fail 2 / ผ่าน 4	Step 6 (2 ข้อที่กดปุ่มขอเมนู)

ทำไมต้องมีโหมดสาธิต

- ให้ดู UI ได้ทันทีโดยไม่ต้องมี key / ไม่ต้องต่อเน็ต และไม่กินโควตา
- ให้เทสต์รันได้เร็วและเหมือนเดิมทุกครั้ง (AI จริงตอบไม่เหมือนเดิม จึงเทสต์อัตโนมัติยาก)
- ในโค้ดทำด้วย `abstract class AiService` ที่มี 2 implementation: `GeminiAiService` (จริง) และ `DemoAiService` (จำลอง) หน้าจอไม่รู้เลยว่าคุยกับตัวไหน — เทคนิคเดียวกับ FakeApiService ใน LabStock

✔ Checkpoint 2

- แอปรันบนอีมูเลเตอร์ เปิดโหมดสาธิตแล้วเห็นหน้าตั้งค่าตอบข้อความจำลอง
- `flutter test` ได้ผล 5 passed, 7 failed (ตัวเลขนี้คือจุดเริ่มต้น)

STEP 3 · อ่าน+ทำตาม

ทัวร์โค้ดที่ให้มา: ข้อมูลไหลจากหน้าจอไปถึง Gemini อย่างไร

เวลาโดยประมาณ 30 นาที

เป้าหมาย: วาดภาพในหัวได้ว่าแต่ละไฟล์ทำหน้าที่อะไร และจุดที่ต้องเขียนเองอยู่ตรงไหนของเส้นทาง

แอปแบ่งเป็น 3 ชั้น เหมือน LabStock (screens → AppState/ApiService → เซิร์ฟเวอร์) เปลี่ยนแค่ปลายทางจาก PHP เป็น Gemini:

```
หน้าจอ (screens/)          งานของแอป (data/ai_service.dart)       คุยกับ Gemini (data/gemin
i_client.dart)
┌─────────────────────┐   ┌─────────────────────────────┐   ┌────────────────────────────
──────┐
│ IngredientsScreen   │──▶│ AiService.suggestMenus(list) │──▶│ GeminiClient.generate(
│
│  "ไข่, หมูสับ"        │   │ • ประกอบ prompt + schema      │   │  prompt, systemInstructio
n, │   HTTPS
│                     │◀──│  • แปลง JSON → MenuResult     │◀──│   imageBytes?, responseS
chema)│◀──────────▶ Gemini API
│ PhotoScreen         │──▶│ AiService.analyzePhoto(bytes)│──▶│  • ประกอบ body JSON
│
│  รูป (bytes)          │   │  • แปลง JSON → PhotoAnalysis  │   │ • POST + header x-goog-
api-key│
└─────────────────────┘   └─────────────────────────────┘   │  • ตรวจ status, แกะ text
│
                                   ▲                           └───────────────────────
──────┘
                              AppSettings (data/app_settings.dart) เลือกว่าใช้ GeminiAiService
หรือ DemoAiService
```

ไฟล์	หน้าที่	สถานะใน starter
`lib/main.dart`	สร้าง AppSettings โหลดค่าจากเครื่อง แล้วเปิด HomeScreen	เสร็จ
`lib/theme.dart`	สีและสไตล์ปุ่ม/ช่องกรอก	เสร็จ
`lib/data/ai_config.dart`	ชื่อโมเดล, URL, timeout — ค่าคงที่ที่อาจต้องแก้ในอนาคต	เสร็จ
`lib/data/gemini_client.dart`	ส่ง HTTP ไป Gemini และแกะคำตอบ (ไม่รู้จักเรื่องอาหารเลย)	TODO Step 5
`lib/data/prompts.dart`	ข้อความ prompt และ JSON schema ที่ให้ AI ตอบ	เสร็จ (จะได้ปรับใน Step 6)
`lib/data/ai_service.dart`	งานของแอป: suggestMenus / analyzePhoto / ping (+ โหมดสาธิต)	TODO Step 6, 7

`lib/data/app_settings.dart`	เก็บ key + โหมดสาธิต, สร้าง AiService ให้หน้าจอ	เสร็จ
`lib/models/*.dart`	คลาสข้อมูล MenuSuggestion, MenuResult, FoundItem, PhotoAnalysis (fromJson)	เสร็จ
`lib/screens/home_screen.dart`	หน้าแรก + แบนเนอร์เตือน key	เสร็จ
`lib/screens/settings_screen.dart`	ใส่ key, โหมดสาธิต, ปุ่มทดสอบ	เสร็จ
`lib/screens/ingredients_screen.dart`	chips วัตถุดิบ → ขอเมนู → MenuCard	TODO Step 6 (_ask)
`lib/screens/photo_screen.dart`	เลือกรูป → วิเคราะห์ → ส่งต่อ	TODO Step 7 (_pickImage, _analyze)
`lib/widgets/*.dart`	MenuCard (การ์ดขยายได้), ErrorBox (error + ลองใหม่)	เสร็จ

3.1 ค่าคงที่: ai_config.dart

`lib/data/ai_config.dart` ไฟล์เต็ม

```dart
/// ค่าคงที่สำหรับติดต่อ Gemini API — แก้ที่ไฟล์นี้ที่เดียว
///
/// โมเดลที่ใช้ได้ฟรี (ดูรายการล่าสุดที่ https://ai.google.dev/gemini-api/docs/models)
/// ถ้าโดนจำกัดโควตา (429) ลองเปลี่ยนเป็น 'gemini-3.5-flash-lite'
const String kGeminiModel = 'gemini-3.8-flash';

/// URL หลักของ Gemini API (เวอร์ชัน v1beta)
const String kGeminiBaseUrl = 'https://generativelanguage.googleapis.com/v1beta';

/// ระดับ "การคิดก่อนตอบ" ของโมเดล Gemini 3.x: 'low' | 'medium' | 'high'
/// ค่าเริ่มต้นของ Google คือ medium ซึ่งช้า (บางครั้งเกิน 1 นาที) งานแนะนำเมนูใช้ low ก็พอ
/// ตั้งเป็น '' เพื่อไม่ส่งค่านี้ (ใช้ค่าเริ่มต้นของโมเดล)
const String kThinkingLevel = 'low';

/// เวลาสูงสุดที่รอ AI ตอบ
const Duration kAiTimeout = Duration(seconds: 90);

/// สำหรับทดสอบเร็ว ๆ ใส่ key ตรงนี้ได้ แต่ **ห้าม commit ขึ้น git เด็ดขาด**
/// (ปกติให้ใส่ผ่านหน้า "ตั้งค่า" ในแอป ซึ่งเก็บไว้ในเครื่องเท่านั้น)
const String kDevApiKey = '';
```

ชื่อโมเดลของ Google เปลี่ยนทุกไม่กี่เดือน ถ้าวันหนึ่งแอปขึ้น error 404 "ไม่พบโมเดล" ให้เปิดหน้า https://ai.google.dev/gemini-api/docs/models เลือกโมเดลตระกูล Flash ที่ยังอยู่ แล้วแก้ที่บรรทัดเดียวนี้

3.2 โมเดลข้อมูล: แปลง JSON เป็น object

lib/models/menu_suggestion.dart ส่วน MenuSuggestion

```
class MenuSuggestion {
  MenuSuggestion({
    required this.name,
    required this.description,
    required this.ingredients,
    required this.missing,
    required this.steps,
    required this.time,
    required this.difficulty,
  });

  final String name;
  final String description;
  final List<String> ingredients; // วัตถุดิบที่ใช้ (พร้อมปริมาณ)
  final List<String> missing; // ต้องซื้อเพิ่ม
  final List<String> steps; // ขั้นตอนทำ
  final String time;
  final String difficulty;

  factory MenuSuggestion.fromJson(Map<String, dynamic> json) => MenuSuggestion(
        name: (json['name'] ?? '').toString(),
        description: (json['description'] ?? '').toString(),
        ingredients: _stringList(json['ingredients']),
        missing: _stringList(json['missing']),
        steps: _stringList(json['steps']),
        time: (json['time'] ?? '-').toString(),
        difficulty: (json['difficulty'] ?? '-').toString(),
      );
```

รูปแบบเดียวกับ `Equipment.fromJson` ใน LabStock ต่างตรงที่ JSON มาจาก AI ไม่ใช่ฐานข้อมูล จึง**ป้องกันค่าหายทุกช่อง** (`?? ''`, `?? const []`) เพราะ AI อาจตอบไม่ครบแม้เราจะกำหนด schema ไว้ก็ตาม

3.3 การตั้งค่า: app_settings.dart

lib/data/app_settings.dart ส่วนบน

```
class AppSettings extends ChangeNotifier {
  static const _keyApiKey = 'gemini_api_key';
  static const _keyDemo = 'demo_mode';

  String _apiKey = kDevApiKey;
  bool _demoMode = false;
  bool _loaded = false;

  String get apiKey => _apiKey;
  bool get demoMode => _demoMode;
  bool get loaded => _loaded;
  bool get hasApiKey => _apiKey.trim().isNotEmpty;

  /// พร้อมใช้ AI หรือยัง (มี key หรือเปิดโหมดสาธิต)
  bool get ready => demoMode || hasApiKey;

  /// สร้าง AiService ตามการตั้งค่าปัจจุบัน
  AiService get aiService =>
      _demoMode ? DemoAiService() : GeminiAiService(GeminiClient(apiKey: _apiKey));
```

จุดสำคัญในไฟล์นี้

- `aiService` สร้าง `GeminiAiService(GeminiClient(apiKey: _apiKey))` ใหม่ทุกครั้งที่ถูกเรียก จึงได้ key ล่าสุดเสมอ ไม่ต้องรีสตาร์ตแอปหลังเปลี่ยน key
- `ready` = มี key หรือเปิดโหมดสาธิต → HomeScreen ใช้ตัดสินใจว่าจะโชว์แถบเตือนไหม
- เป็น `ChangeNotifier` และ HomeScreen ครอบด้วย `ListenableBuilder` → บันทึกค่าปุ๊บหน้าแรกวาดใหม่ปั๊บ

3.4 หน้าจอเรียก AI อย่างไร (ดูจากหน้าตั้งค่าที่เสร็จแล้ว)

`lib/screens/settings_screen.dart` เมธอด `_test()`

```dart
/// ส่งคำถามสั้น ๆ เพื่อเช็กว่า key ใช้ได้
Future<void> _test() async {
  setState(() {
    _testing = true;
    _testResult = null;
  });
  // บันทึกค่าที่พิมพ์ไว้ก่อน แล้วค่อยสร้าง service จากค่าล่าสุด
  await widget.settings.save(apiKey: _keyCtrl.text, demoMode: _demo);
  try {
    final reply = await widget.settings.aiService.ping();
    if (!mounted) return;
    setState(() {
      _testOk = true;
      _testResult = reply;
    });
  } on AiException catch (e) {
    if (!mounted) return;
    setState(() {
      _testOk = false;
      _testResult = e.message;
    });
  } finally {
    if (mounted) setState(() => _testing = false);
  }
}
```

นี่คือ**แม่แบบ**ของทุกเมธอดที่คุยกับ AI ในแอป จำโครงนี้ไว้ จะได้ใช้ซ้ำใน Step 6 และ 7:

1. setState เปิดสถานะ "กำลังโหลด" และล้างผลเก่า
2. `await` เรียก AiService ใน `try`
3. `on AiException` → เก็บ `e.message` (ข้อความภาษาไทยพร้อมแสดง)
4. `finally` → ปิดสถานะโหลด
5. **เช็ก `mounted` ก่อน setState ทุกครั้งหลัง await** — ผู้ใช้อาจกดย้อนกลับไปแล้วระหว่างรอ AI (ซึ่งนานหลายวินาที)

✔ Checkpoint 3

- บอกได้ว่าเมื่อผู้ใช้กด "ให้ AI แนะนำเมนู" โค้ดจะวิ่งผ่านไฟล์ไหนบ้างตามลำดับ
- เปิดไฟล์ 4 ไฟล์ที่มี TODO และอ่านคอมเมนต์ TODO ทุกจุดแล้ว

STEP 4 · อ่าน+ทำตาม

เข้าใจ Gemini API: JSON ที่ส่งไป และ JSON ที่ได้กลับมา

เวลาโดยประมาณ 25 นาที

เป้าหมาย: อ่าน/เขียน request และ response ของ generateContent ได้ทุกช่อง ก่อนลงมือเขียนโค้ด

Gemini มี REST API ชื่อ `generateContent` ทำงานแบบ "ส่ง JSON ไป ได้ JSON กลับ" ครั้งเดียวจบ (ไม่จำบทสนทนา — ถ้าอยากคุยต่อเนื่องต้องส่งประวัติไปเองทุกครั้ง ดูโจทย์ท้าทายข้อ B) URL มีรูปแบบคงที่:

```
POST https://generativelanguage.googleapis.com/v1beta/models/{ชื่อโมเดล}:generateContent
Header  x-goog-api-key: <API key>
Header  Content-Type: application/json
```

4.1 Request: สิ่งที่เราส่งไป

```
{
  "system_instruction": {                       // (ไม่บังคับ) บทบาท/กติกาที่ AI ต้องยึดตลอด
    "parts": [ { "text": "คุณคือเชฟอาหารไทย ตอบเป็นภาษาไทย ..." } ]
  },
  "contents": [                                 // บทสนทนา — แล็บนี้ส่งแค่ 1 ข้อความจากผู้ใช้
    {
      "role": "user",
      "parts": [                                // 1 ข้อความมีหลาย "ชิ้น" ได้: ข้อความ + รูป
        { "text": "ตอนนี้ฉันมี ไข่, หมูสับ ..." },
        { "inline_data": { "mime_type": "image/jpeg", "data": "<รูปเข้ารหัส base64>" } }
      ]
    }
  ],
  "generationConfig": {                         // ตัวเลือกการตอบ
    "temperature": 0.7,                         // 0 = ตอบนิ่ง ๆ เหมือนเดิม · 1–2 = สร้างสรรค์/สุ่ม
มากขึ้น
    "maxOutputTokens": 8192,                    // เพดานความยาวคำตอบ
    "thinkingConfig": { "thinkingLevel": "low" },// Gemini 3.x "คิดก่อนตอบ" ค่าเริ่มต้น medium ช้า
มาก → ลดเป็น low
    "responseMimeType": "application/json",     // (ไม่บังคับ) บังคับให้ตอบเป็น JSON ล้วน
    "responseSchema": { "type": "object", ... } // (ไม่บังคับ) โครงสร้าง JSON ที่ต้องการ
  }
}
```

ช่อง	เขียนอย่างไร	ทำไมสำคัญ
`system_instruction`	ข้อความบอกบทบาท กติกา ภาษา รูปแบบ	เขียนครั้งเดียวใช้กับทุกคำถาม ทำให้คำตอบนิ่งและเป็นแนวเดียวกัน (ของเราอยู่ใน prompts.dart)
`contents[].parts[]`	อาร์เรย์ของ `{text}` หรือ `{inline_data}`	ส่งรูปกับข้อความไปพร้อมกันได้ = multimodal · รูปต้องเป็น base64 ไม่ใช่ URL

`temperature`	0–2	งาน "ดูรูปแล้วบอกว่ามีอะไร" ใช้ค่าต่ำ (0.2) · งาน "คิดเมนู" ใช้ 0.7 ให้มีความหลากหลาย
`thinkingConfig.thinkingLevel`	`low` / `medium` / `high`	โมเดล Gemini 3.x จะ "คิดในใจ" ก่อนตอบ ยิ่งระดับสูงยิ่งฉลาดแต่ช้าและกิน token — งานแนะนำเมนูใช้ low พอ (ไม่ใส่ = medium ซึ่งเคยรอเกิน 90 วินาที)
`responseSchema`	JSON Schema แบบย่อ (type/properties/items/required/enum)	ไม่ต้องเขียน regex แกะคำตอบ — AI ตอบตรงโครงสร้าง แปลงด้วย `jsonDecode` ได้เลย

> **ทำไม key ต้องอยู่ใน header ไม่ใช่ใน URL**
>
> - เอกสารเก่าของ Google ใส่ `?key=...` ท้าย URL ซึ่งใช้ได้ แต่ URL มักถูกบันทึกใน log ของเซิร์ฟเวอร์ proxy และประวัติต่าง ๆ ทำให้ key รั่วง่าย
> - header `x-goog-api-key` เป็นวิธีที่ Google แนะนำในปัจจุบัน และเทสต์ของเราตรวจว่า key **ต้องไม่**อยู่ใน URL

4.2 Response: สิ่งที่ได้กลับมา

```
{
  "candidates": [                                // คำตอบ (ปกติมี 1 ตัว)
    {
      "content": { "role": "model", "parts": [ { "text": "{\"summary\": \"...\", \"menus\": [...]}" } ] },
      "finishReason": "STOP"                    // STOP · MAX_TOKENS · SAFETY · RECITATION ...
    }
  ],
  "promptFeedback": { "blockReason": "SAFETY" },// มีเฉพาะตอน "คำถาม" ถูกบล็อก (แล้ว candidates จะว่าง)
  "usageMetadata": { "promptTokenCount": 310, "candidatesTokenCount": 820, "totalTokenCount": 1130 },
  "modelVersion": "gemini-3.8-flash"
}
```

สังเกตว่าเมื่อเราขอ `responseMimeType: application/json` ค่าใน `text` จะเป็น**สตริง JSON** (JSON ซ้อนใน JSON) เราจึงต้อง `jsonDecode` สองชั้น: ชั้นแรกแกะ response ทั้งก้อน (ทำใน GeminiClient) ชั้นสองแกะข้อความคำตอบเป็น MenuResult (ทำใน AiService)

4.3 เมื่อผิดพลาด

HTTP	body ที่ได้	สาเหตุที่พบบ่อย	แอปควรบอกผู้ใช้ว่า

400	`{"error":{"code":400,"message":"API key not valid..."}}`	key ผิด / คัดลอกไม่ครบ / ถูกลบไปแล้ว	ตรวจ key ที่หน้าตั้งค่า
400	`"message":"Invalid JSON payload..."`	body ผิดรูปแบบ (สะกดชื่อช่องผิด)	บั๊กของเรา ดู message แล้วแก้
403	`"status":"PERMISSION_DENIED"`	key ไม่มีสิทธิ์ใช้ API นี้ / ประเทศไม่รองรับ	สร้าง key ใหม่
404	`"status":"NOT_FOUND"`	ชื่อโมเดลผิดหรือถูกปลดระวาง	แก้ kGeminiModel
429	`"status":"RESOURCE_EXHAUSTED"`	ยิงเกินโควตาฟรีต่อนาที/ต่อวัน	รอสักครู่ · เปลี่ยนไปโมเดล lite
503	`"status":"UNAVAILABLE"`	เซิร์ฟเวอร์ Google คนเยอะ	ลองใหม่
200 แต่ candidates ว่าง	`"promptFeedback": {"blockReason":"SAFETY"}`	คำถาม/รูปเข้าข่ายไม่เหมาะสม	เปลี่ยนคำถาม

ทุกกรณีในตารางถูกแปลงเป็นภาษาไทยไว้แล้วใน `GeminiClient.describeHttpError()` (ให้มาใน starter) หน้าที่ของคุณใน Step 5 คือ**เรียกใช้**มันให้ถูกจังหวะ

✔ Checkpoint 4

- เขียน body JSON สำหรับถามว่า "ไข่ 2 ฟอง ทำอะไรได้บ้าง" ด้วยมือได้โดยไม่เปิดดู
- บอกได้ว่า JSON ซ้อน 2 ชั้นคืออะไร และแต่ละชั้นถูกแกะที่ไฟล์ไหน

STEP 5 · ทำเอง

เขียน GeminiClient: ส่งคำขอและแกะคำตอบ

เวลาโดยประมาณ 50 นาที

เป้าหมาย: generate() ยิง HTTP ไป Gemini ได้จริง · extractText() แกะข้อความได้ · เทสต์ 6 ข้อผ่าน · ปุ่ม "ทดสอบการเชื่อมต่อ" ได้คำตอบจาก Gemini จริง

เปิด `lib/data/gemini_client.dart` ใน starter คุณจะเห็นโครงนี้ (ตัดส่วน helper ออก):

`lib/data/gemini_client.dart` starter · ส่วนที่ต้องเขียน

```
Future<String> generate({
  required String prompt,
  String? systemInstruction,
  Uint8List? imageBytes,
  String imageMimeType = 'image/jpeg',
  Map<String, dynamic>? responseSchema,
  double temperature = 0.7,
}) async {
  if (apiKey.trim().isEmpty) {
    throw AiException('ยังไม่ได้ตั้งค่า API key — ไปที่หน้า "ตั้งค่า" ก่อน');
  }

  // TODO(Step 5.1): ประกอบ parts = [ {'text': prompt}, (ถ้ามีรูป) {'inline_data': {...}} ]

  // TODO(Step 5.2): ประกอบ body ตามรูปแบบของ Gemini
  //   system_instruction (ถ้ามี) · contents · generationConfig (temperature, maxOutputTokens,
  //   thinkingConfig.thinkingLevel = kThinkingLevel, และถ้ามี responseSchema ให้ใส่ responseMimeType + responseSchema)

  // TODO(Step 5.3): ยิง HTTP POST ไปที่ endpoint พร้อม header 'x-goog-api-key'
  //   ครอบด้วย try/catch: TimeoutException, SocketException, http.ClientException → AiException

  // TODO(Step 5.4): ถ้า statusCode != 200 → throw AiException(describeHttpError(...), statusCode)

  // TODO(Step 5.5): แกะข้อความด้วย extractText(jsonDecode(res.body))
  throw UnimplementedError('ยังไม่ได้เขียน generate() — ทำตามคู่มือ Step 5');
}
```

สิ่งที่ให้มาแล้วในไฟล์: คลาส `AiException`, constructor ที่รับ `http.Client` (เพื่อให้เทสต์สวมเซิร์ฟเวอร์ปลอมได้), `endpoint`, `describeHttpError()` และการเช็ก key ว่าง ลบคอมเมนต์ TODO และบรรทัด `throw UnimplementedError` ออก แล้วเขียนทีละข้อตามนี้

5.1 ประกอบ parts

lib/data/gemini_client.dart 1/5

```
// 1) ประกอบ "ชิ้นส่วน" ของคำถาม: ข้อความ และรูป (ถ้ามี)
final parts = <Map<String, dynamic>>[
  {'text': prompt},
  if (imageBytes != null)
    {
      'inline_data': {'mime_type': imageMimeType, 'data': base64Encode(imageBytes)},
    },
];
```

- ใช้ collection if (`if (imageBytes != null) {...}` ข้างใน list) จะได้ไม่ต้องสร้าง list แล้ว add ทีหลัง
- `base64Encode` มาจาก `dart:convert` แปลง bytes ของรูปเป็นข้อความ ASCII ยาว ๆ — รูป 100 KB จะกลายเป็นข้อความ ~133 KB นี่คือเหตุผลที่หน้ารูปต้องย่อรูปก่อนส่ง

5.2 ประกอบ body

lib/data/gemini_client.dart 2/5

```
// 2) ประกอบ body ตามรูปแบบที่ Gemini ต้องการ
final body = <String, dynamic>{
  if (systemInstruction != null)
    'system_instruction': {
      'parts': [
        {'text': systemInstruction},
      ],
    },
  'contents': [
    {'role': 'user', 'parts': parts},
  ],
  'generationConfig': {
    'temperature': temperature,
    'maxOutputTokens': 8192,
    // ลดเวลาคิดของโมเดล (ไม่ใส่ = medium ซึ่งช้ากว่ามาก)
    if (kThinkingLevel.isNotEmpty) 'thinkingConfig': {'thinkingLevel': kThinkingLevel},
    if (responseSchema != null) 'responseMimeType': 'application/json',
    if (responseSchema != null) 'responseSchema': responseSchema,
  },
};
```

- เทียบกับ JSON ใน Step 4.1 ทีละช่อง ชื่อช่องต้องสะกดเป๊ะ (Gemini รับทั้ง `system_instruction` และ `systemInstruction` แต่ `generationConfig` ต้องเป็น camelCase)
- `thinkingConfig` อ่านจาก `kThinkingLevel` ใน ai_config.dart — ถ้าตอบช้าหรือเร็วไป แก้ที่เดียว
- ใช้ collection if อีกครั้งเพื่อใส่ `responseMimeType` / `responseSchema` เฉพาะตอนที่ผู้เรียกส่ง schema มา — ถ้าใส่ `responseSchema: null` ไป Gemini จะตอบ 400

5.3 ยิง HTTP POST และดักข้อผิดพลาดระดับเครือข่าย

lib/data/gemini_client.dart 3/5

```
// 3) ยิง HTTP POST — key ส่งใน header ไม่ใส่ใน URL
http.Response res;
try {
  res = await _client
      .post(
        endpoint,
        headers: {'Content-Type': 'application/json', 'x-goog-api-key': apiKey},
        body: jsonEncode(body),
      )
      .timeout(kAiTimeout);
} on TimeoutException {
  throw AiException('AI ตอบช้าเกินไป (เกิน ${kAiTimeout.inSeconds} วินาที) ลองใหม่อีกครั้ง');
} on SocketException {
  throw AiException('เชื่อมต่ออินเทอร์เน็ตไม่ได้ ตรวจสอบ Wi-Fi หรือเน็ตมือถือ');
} on http.ClientException catch (e) {
  throw AiException('ส่งคำขอไม่สำเร็จ: ${e.message}');
}
```

- `jsonEncode(body)` แปลง Map เป็นสตริง JSON · header `Content-Type` บอกว่า body เป็น JSON
- `.timeout(kAiTimeout)` สำคัญมาก: AI ใช้เวลาตอบ 3–20 วินาที ถ้าเน็ตหลุดกลางทาง ไม่มี timeout แอปจะหมุนค้างตลอดกาล
- ดัก 3 แบบ: หมดเวลา (`TimeoutException` จาก dart:async) · ไม่มีเน็ต (`SocketException` จาก dart:io) · อื่น ๆ ของ package http (`http.ClientException`) แล้ว**แปลงทั้งหมดเป็น AiException** หน้าจอจึงดักแค่ชนิดเดียว

5.4 ตรวจสถานะ HTTP

lib/data/gemini_client.dart 4/5

```
// 4) ตรวจสถานะ HTTP
if (res.statusCode != 200) {
  throw AiException(describeHttpError(res.statusCode, res.body), res.statusCode);
}
```

5.5 แกะข้อความ

lib/data/gemini_client.dart 5/5

```
  // 5) แกะข้อความออกจาก JSON ที่ตอบกลับ
  return extractText(jsonDecode(res.body) as Map<String, dynamic>);
}
```

5.6 extractText(): เดินเข้าไปใน JSON ให้ปลอดภัย

lib/data/gemini_client.dart ทั้งเมธอด

```dart
  static String extractText(Map<String, dynamic> json) {
    final feedback = json['promptFeedback'] as Map<String, dynamic>?;
    if (feedback?['blockReason'] != null) {
      throw AiException('AI ปฏิเสธคำถามนี้ (${feedback!['blockReason']}) ลองเปลี่ยนคำถาม');
    }
    final candidates = json['candidates'] as List<dynamic>?;
    if (candidates == null || candidates.isEmpty) {
      throw AiException('AI ไม่ได้ส่งคำตอบกลับมา');
    }
    final first = candidates.first as Map<String, dynamic>;
    final parts = (first['content']?['parts'] as List<dynamic>?) ?? const [];
    final text = parts
        .map((p) => (p as Map<String, dynamic>)['text'])
        .whereType<String>()
        .join();
    if (text.trim().isEmpty) {
      final reason = first['finishReason'] ?? 'ไม่ทราบสาเหตุ';
      throw AiException('AI ตอบกลับมาว่างเปล่า (finishReason: $reason)');
    }
    return text;
  }
}
```

อ่านโค้ด extractText ทีละบรรทัด

- ทุกชั้นใช้ `as ...?` (nullable cast) แล้วเช็ก null — เพราะ JSON จาก AI ไม่มีการันตีว่าช่องไหนจะมี การ cast ตรง ๆ แบบ `json['candidates'][0]` จะพังทันทีที่ช่องหาย
- `first['content']?['parts']` — เครื่องหมาย `?` หลัง `['content']` คือ null-aware index: ถ้า content เป็น null ให้ทั้งนิพจน์เป็น null แทนที่จะ crash
- `.whereType<String>()` กรองเอาเฉพาะ part ที่เป็นข้อความ (part ชนิดอื่นเช่น thought/functionCall จะถูกข้าม) แล้ว `.join()` ต่อกัน
- ลำดับการเช็ก: ถูกบล็อก → ไม่มี candidates → ข้อความว่าง แต่ละกรณีให้ข้อความ error ต่างกัน ผู้ใช้จะได้รู้ว่าควรทำอะไรต่อ

5.7 ทดสอบด้วยเซิร์ฟเวอร์ปลอม

ไฟล์ `test/gemini_client_test.dart` ใช้ `MockClient` จาก package http สวมเป็น Gemini — มันจับ request ที่โค้ดคุณสร้างมาตรวจทีละช่อง และตอบ JSON ที่เรากำหนดเองได้ (รวมถึง 429) จึงทดสอบทุกเงื่อนไขได้โดยไม่ใช้โควตาแม้แต่ token เดียว

test/gemini_client_test.dart เทสต์ข้อแรก

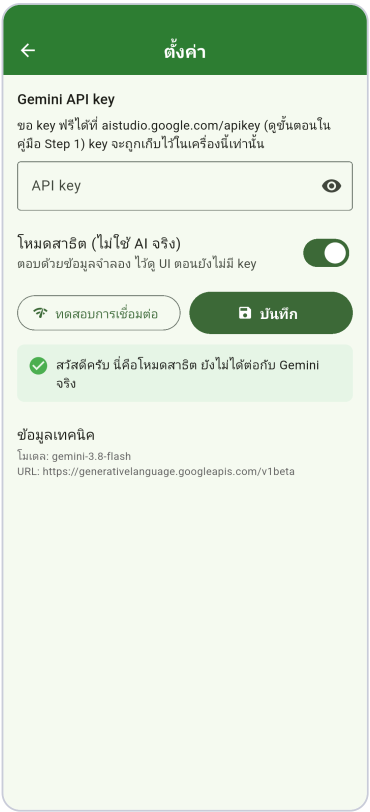
```dart
test('ส่ง header x-goog-api-key และ body ตามรูปแบบ Gemini', () async {
  final (client, seen) = fakeClient(200, okBody);
  final text = await client.generate(prompt: 'ทดสอบ', systemInstruction: 'คุณคือเชฟ');

  expect(text, 'สวัสดีครับ'); // รวมข้อความจากทุก part
  expect(seen, hasLength(1));
  final req = seen.single;
  expect(req.method, 'POST');
  expect(req.url.toString(), contains(':generateContent'));
  expect(req.url.queryParameters.containsKey('key'), isFalse); // key ไม่หลุดไปอยู่ใน URL
  expect(req.headers['x-goog-api-key'], 'test-key');

  final body = jsonDecode(req.body) as Map<String, dynamic>;
  expect(body['contents'][0]['parts'][0]['text'], 'ทดสอบ');
  expect(body['system_instruction']['parts'][0]['text'], 'คุณคือเชฟ');
  expect(body['generationConfig'].containsKey('responseSchema'), isFalse);
  expect(body['generationConfig']['thinkingConfig']['thinkingLevel'], 'low'); // ลดเวลารอ
});
```

```
flutter test test/gemini_client_test.dart
```

ต้องได้ **All tests passed!** (6 ข้อ) ถ้าข้อไหน fail ให้อ่านบรรทัด `Expected:` / `Actual:` ในผลลัพธ์ มันบอกชัดว่าช่องไหนของ JSON ไม่ตรง

5.8 ทดสอบกับ Gemini จริง

1. รันแอป → ตั้งค่า → **ปิด**โหมดสาธิต → วาง API key → กด **ทดสอบการเชื่อมต่อ**
2. ควรได้ประโยคทักทายภาษาไทยพร้อมชื่อโมเดลภายใน 2–5 วินาที (เมธอด `ping()` ใน ai_service.dart เรียก `generate()` ที่คุณเพิ่งเขียน)
3. ลองพิมพ์ key ผิด 1 ตัวแล้วทดสอบอีกครั้ง → ต้องได้ข้อความ "API key ไม่ถูกต้อง..." ไม่ใช่แอป crash
4. ปิด Wi-Fi ของอีมูเลเตอร์ (แถบด้านข้าง → ... → Cellular/Wi-Fi) แล้วทดสอบ → ต้องได้ "เชื่อมต่ออินเทอร์เน็ตไม่ได้"



Gemini API key

ขอ key ฟรีได้ที่ aistudio.google.com/apikey (ดูขั้นตอนในคู่มือ Step 1) key จะถูกเก็บไว้ในเครื่องนี้เท่านั้น

โหมดสาธิต (ไม่ใช้ AI จริง)
ตอบด้วยข้อมูลจำลอง ไว้ดู UI ตอนยังไม่มี key

สวัสดีครับ นี่คือโหมดสาธิต ยังไม่ได้ต่อกับ Gemini จริง

ข้อมูลเทคนิค
โมเดล: gemini-3.8-flash
URL: https://generativelanguage.googleapis.com/v1beta

ผลทดสอบการเชื่อมต่อ (ภาพจากโหมดสาธิต — ของจริงจะเป็นคำทักทายจาก Gemini)

✔ Checkpoint 5

- `flutter test test/gemini_client_test.dart` ผ่าน 6/6
- ปุ่มทดสอบการเชื่อมต่อได้คำตอบจาก Gemini จริง (ถ่ายภาพหน้าจอไว้)
- key ผิดและไม่มีเน็ตได้ข้อความภาษาไทยที่ถูกต้อง ไม่ crash

STEP 6 · ทำเอง

หน้า "เมนูจากวัตถุดิบ": prompt, JSON schema และการแสดงผล

เวลาโดยประมาณ 45 นาที

เป้าหมาย: พิมพ์วัตถุดิบ → กดปุ่ม → ได้การ์ดเมนู 3 รายการจาก Gemini จริง พร้อมวัตถุดิบ ของที่ต้องซื้อ และวิธีทำ

6.1 อ่าน prompt ที่ให้มา (prompts.dart)

`lib/data/prompts.dart` system instruction + buildMenuPrompt

```dart
const String kChefSystemInstruction = '''
คุณคือเชฟอาหารไทยที่ใจดี ชอบสอนคนเริ่มทำอาหาร
- ตอบเป็นภาษาไทยที่อ่านง่าย ใช้ชื่อวัตถุดิบและหน่วยแบบไทย (ช้อนโต๊ะ, ถ้วย, กรัม)
- แนะนำเมนูที่ทำได้จริงในครัวบ้านทั่วไป ใช้เวลาไม่นาน
- ถ้าวัตถุดิบไม่พอ ให้บอกชัดเจนว่าต้องซื้ออะไรเพิ่ม (ของพื้นฐานอย่างน้ำปลา น้ำตาล น้ำมัน กระเทียม ถือว่ามีอยู่แล้ว)
''';

/// สร้างคำถามสำหรับขอเมนูจากรายการวัตถุดิบ
String buildMenuPrompt(List<String> ingredients, {String? extra}) {
  final list = ingredients.map((e) => '- $e').join('\n');
  final condition = (extra == null || extra.trim().isEmpty) ? '' : '\nเงื่อนไขเพิ่มเติม: ${extra.trim()}';
  return '''
ตอนนี้ฉันมีวัตถุดิบดังนี้:
$list
$condition
ช่วยแนะนำเมนูที่ทำได้ 3 เมนู เรียงจากเมนูที่ใช้ของที่มีอยู่ครบที่สุดก่อน
แต่ละเมนูบอก: ชื่อเมนู, คำอธิบายสั้น ๆ, วัตถุดิบที่ใช้ (พร้อมปริมาณ), ของที่ต้องซื้อเพิ่ม, ขั้นตอนการทำทีละข้อ, เวลาที่ใช้, ความยาก
''';
}

/// โครงสร้าง JSON ที่ให้ AI ตอบสำหรับ "แนะนำเมนู" (JSON Schema แบบย่อที่ Gemini รองรับ)
```

หลักเขียน prompt ที่ใช้ในไฟล์นี้

- **บทบาท** ("คุณคือเชฟ...") ทำให้น้ำเสียงและความรู้ที่ดึงมาใช้ตรงงาน
- **กติกาเป็นข้อ ๆ** อ่านง่ายกว่าย่อหน้ายาว ๆ ทั้งคนและ AI
- **บอกข้อสมมติ** (ของพื้นฐานถือว่ามี) ลดการตอบว่า "ต้องซื้อน้ำปลา" ซึ่งไม่มีประโยชน์
- **ระบุจำนวนและลำดับ** (3 เมนู เรียงจากของครบที่สุด) คำตอบจะคงที่ เอาไปแสดงผลได้
- **แยก system กับ user:** ของที่คงที่ทุกครั้งไว้ใน system_instruction · ของที่เปลี่ยนตามผู้ใช้ (รายการวัตถุดิบ เงื่อนไข) ไว้ใน prompt

`lib/data/prompts.dart` — JSON schema ของคำตอบ

```dart
const Map<String, dynamic> kMenuResponseSchema = {
  'type': 'object',
  'properties': {
    'summary': {'type': 'string', 'description': 'สรุปสั้น ๆ 1–2 ประโยคว่าวัตถุดิบชุดนี้เหมาะทำอะไร'},
    'menus': {
      'type': 'array',
      'items': {
        'type': 'object',
        'properties': {
          'name': {'type': 'string'},
          'description': {'type': 'string'},
          'ingredients': {
            'type': 'array',
            'items': {'type': 'string'},
            'description': 'วัตถุดิบพร้อมปริมาณ เช่น "ไข่ไก่ 2 ฟอง"',
          },
          'missing': {
            'type': 'array',
            'items': {'type': 'string'},
            'description': 'ของที่ต้องซื้อเพิ่ม (ว่างได้ถ้าครบ)',
          },
          'steps': {
            'type': 'array',
            'items': {'type': 'string'},
          },
          'time': {'type': 'string', 'description': 'เช่น "15 นาที"'},
          'difficulty': {'type': 'string', 'enum': ['ง่าย', 'ปานกลาง', 'ยาก']},
        },
        'required': ['name', 'description', 'ingredients', 'missing', 'steps', 'time', 'difficulty'],
      },
    },
  },
  'required': ['summary', 'menus'],
};
```

Schema นี้คือ "แบบฟอร์ม" ที่ AI ต้องกรอก สังเกต `description` ในแต่ละช่อง — AI อ่านมันด้วย จึงใช้บอกรูปแบบย่อย ๆ ได้ (เช่น "ไข่ไก่ 2 ฟอง") และ `enum` จำกัดค่าให้เลือกได้แค่ 3 แบบ ซึ่งคลาส `MenuSuggestion.fromJson` รับได้พอดี

6.2 เขียน GeminiAiService.suggestMenus()

`lib/data/ai_service.dart` — เฉลย · ลบ TODO แล้วเขียนแบบนี้

```dart
Future<MenuResult> suggestMenus(List<String> ingredients, {String? extra}) async {
  final text = await client.generate(
    prompt: buildMenuPrompt(ingredients, extra: extra),
    systemInstruction: kChefSystemInstruction,
    responseSchema: kMenuResponseSchema,
  );
  return MenuResult.fromJsonText(text);
}
```

แค่ 3 บรรทัด: ประกอบ prompt → เรียก generate พร้อม schema → แปลงสตริง JSON เป็น MenuResult (`fromJsonText` เรียก `jsonDecode` ให้แล้ว นี่คือ JSON ชั้นที่สองจาก Step 4.2)

6.3 เขียน _ask() ในหน้าจอ

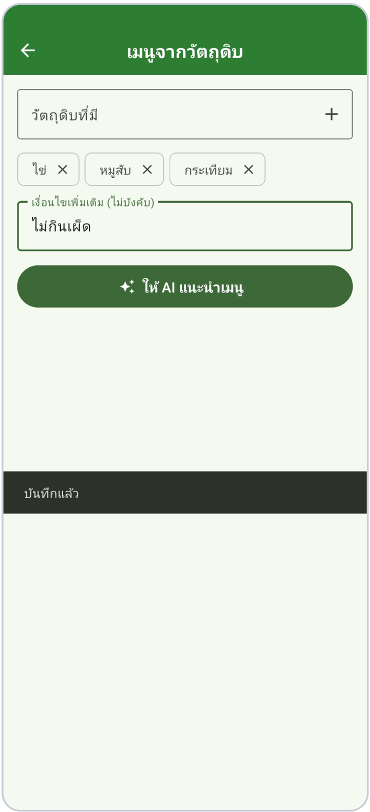
lib/screens/ingredients_screen.dart เฉลย

```dart
/// ส่งรายการวัตถุดิบไปถาม AI
Future<void> _ask() async {
  if (_ingredients.isEmpty) {
    ScaffoldMessenger.of(context)
        .showSnackBar(const SnackBar(content: Text('ใส่วัตถุดิบอย่างน้อย 1 อย่างก่อน')));
    return;
  }
  setState(() {
    _loading = true;
    _error = null;
    _result = null;
  });
  try {
    final result = await widget.ai.suggestMenus(_ingredients, extra: _extraCtrl.text);
    if (!mounted) return;
    setState(() => _result = result);
  } on AiException catch (e) {
    if (!mounted) return;
    setState(() => _error = e.message);
  } catch (e) {
    // เช่น AI ตอบ JSON ผิดรูปแบบ → FormatException
    if (!mounted) return;
    setState(() => _error = 'อ่านคำตอบของ AI ไม่ได้: $e');
  } finally {
    if (mounted) setState(() => _loading = false);
  }
}
```

- โครงเดียวกับ `_test()` ในหน้าตั้งค่า (Step 3.4) เพิ่ม `catch (e)` ท้ายสุดดักกรณี AI ตอบ JSON เพี้ยนจน `jsonDecode` พัง (`FormatException`) — เกิดได้แม้มี schema เช่นตอนคำตอบยาวเกิน maxOutputTokens แล้วถูกตัดกลางคัน
- ส่วน build() ให้มาแล้ว: มันดูตัวแปร 3 ตัว `_loading / _error / _result` แล้วเลือกวาด spinner / ErrorBox / การ์ดเมนู ตามลำดับ — คุณแค่ทำให้ตัวแปร 3 ตัวนี้มีค่าถูกต้อง

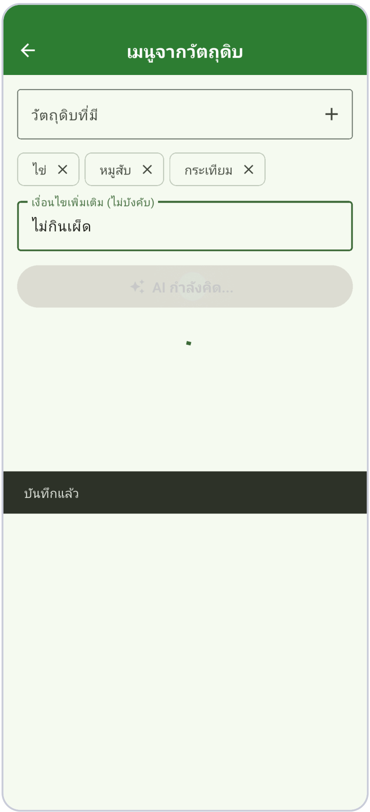
6.4 รันจริง

1. รันแอป (โหมดสาธิตปิด, key ใส่แล้ว) → "พิมพ์วัตถุดิบที่มี" → พิมพ์ `ไข่, หมูสับ, กระเทียม` กด Enter (พิมพ์หลายอย่างคั่นด้วย , ได้) → เงื่อนไข "ไม่กินเผ็ด" → กดปุ่ม
2. รอ 5–20 วินาที (มีตัวนับวินาทีให้ดูว่ายังทำงานอยู่) จะได้ 3 การ์ด การ์ดแรกขยายอยู่ กดการ์ดอื่นเพื่อขยาย สังเกตป้าย "ของครบ" / "ซื้อเพิ่ม n"
3. กดปุ่มซ้ำ → คำตอบไม่เหมือนเดิม (temperature 0.7) ลองแก้เป็น 0 ใน `generate(... temperature: ...)` ดูว่าต่างไหม
4. ลบวัตถุดิบออกให้หมดแล้วกด → ต้องได้ SnackBar เตือน ไม่ยิงคำขอ


เมนูจากวัตถุดิบ
วัตถุดิบที่มี
ไข่
หมูสับ
กระเทียม
เงื่อนไขเพิ่มเติม (ไม่บังคับ)
ให้ AI แนะนำเมนู
บันทึกแล้ว

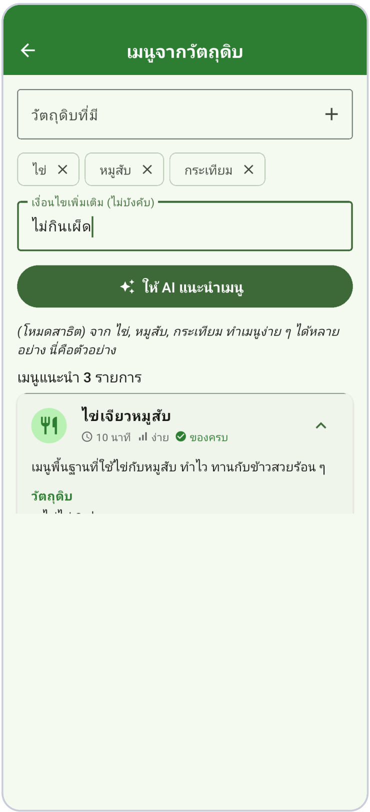
ไม่กินเผ็ด

กรอกวัตถุดิบและเงื่อนไข


เมนูจากวัตถุดิบ
วัตถุดิบที่มี
ไข่
หมูสับ
กระเทียม
เงื่อนไขเพิ่มเติม (ไม่บังคับ)
AI กำลังคิด...
บันทึกแล้ว

ไม่กินเผ็ด

ระหว่างรอ AI


เมนูจากวัตถุดิบ
วัตถุดิบที่มี
ไข่
หมูสับ
กระเทียม
เงื่อนไขเพิ่มเติม (ไม่บังคับ)
ให้ AI แนะนำเมนู
10 นาที
ง่าย
ของครบ
วัตถุดิบ

ไม่กินเผ็ด

(โหมดสาธิต) จาก ไข่, หมูสับ, กระเทียม ทำเมนูง่าย ๆ ได้หลายอย่าง นี่คือตัวอย่าง

เมนูแนะนำ 3 รายการ

ไข่เจียวหมูสับ

เมนูพื้นฐานที่ใช้ไข่กับหมูสับ ทำไว ทานกับข้าวสวยร้อน ๆ

ผลลัพธ์ (ภาพจากโหมดสาธิต)

6.5 ทดลองแก้ prompt (ทำแล้วบันทึกผลส่งด้วย)

ทดลอง	แก้ตรงไหน	คาดว่าจะเห็น
ขอ 5 เมนู	buildMenuPrompt: "3 เมนู" → "5 เมนู"	การ์ด 5 ใบ (ช้าลงเล็กน้อย)
ตอบเป็นอังกฤษ	kChefSystemInstruction บรรทัดภาษา	ชื่อเมนู/วิธีทำเป็นอังกฤษ แต่โครง JSON เหมือนเดิม
เชฟอารมณ์ร้าย	เพิ่มบุคลิกใน system instruction	น้ำเสียงใน description เปลี่ยน — ตัวอย่างว่า system instruction มีอิทธิพลแค่ไหน
ลบ responseSchema ออก	suggestMenus: ไม่ส่ง responseSchema	AI ตอบเป็น markdown ธรรมดา → fromJsonText พัง → เห็น "อ่านคำตอบของ AI ไม่ได้" (นี่คือเหตุผลที่ต้องมี schema)

✔ Checkpoint 6

- `flutter test` ผ่านครบ 12/12 (เทสต์หน้าจอ 2 ข้อที่เคย fail ต้องผ่านแล้ว)
- ได้เมนูจาก Gemini จริงจากวัตถุดิบที่คุณพิมพ์เอง (ถ่ายภาพหน้าจอ 2 ภาพ: ก่อนกดและผลลัพธ์)
- บันทึกผลการทดลองแก้ prompt อย่างน้อย 2 ข้อ (แก้อะไร เห็นอะไร)

STEP 7 · ทำเอง

หน้า "ถ่ายรูปวัตถุดิบ": ส่งรูปให้ AI ดู

เวลาโดยประมาณ 45 นาที

เป้าหมาย: ถ่าย/เลือกรูป → AI บอกว่าเห็นอะไร → กดปุ่มส่งต่อไปขอเมนูจากของในรูป

ตอนนี้ท่อส่งข้อมูลพร้อมแล้ว (generate รับ imageBytes ได้ตั้งแต่ Step 5) เหลือแค่ 3 ชิ้น: เอารูปเข้ามา · ส่งไปวิเคราะห์ · แปลงผล

7.1 เขียน _pickImage(): เอารูปเข้ามาและย่อก่อน

`lib/screens/photo_screen.dart` เฉลย

```dart
/// เปิดกล้องหรือคลังรูป แล้วย่อรูปให้เล็กลงก่อนส่ง (ประหยัดเน็ตและโควตา)
Future<void> _pickImage(ImageSource source) async {
  final XFile? file = await _picker.pickImage(
    source: source,
    maxWidth: 1024,
    maxHeight: 1024,
    imageQuality: 80,
  );
  if (file == null) return; // ผู้ใช้กดยกเลิก
  final bytes = await file.readAsBytes();
  setState(() {
    _imageBytes = bytes;
    _mimeType = file.mimeType ?? _guessMime(file.name);
    _result = null;
    _error = null;
  });
}
```

ทำไมต้องย่อรูป

- กล้องมือถือถ่ายได้รูป 3–8 MB · base64 ทำให้ใหญ่ขึ้นอีก 33% · Gemini จำกัดคำขอทั้งก้อนที่ 20 MB และยิ่งใหญ่ยิ่งช้าและกินโควตา token
- `maxWidth/maxHeight: 1024` + `imageQuality: 80` ให้รูป ~100–300 KB ซึ่ง AI ดูออกสบาย ๆ (หน้าจอแสดงขนาดที่จะส่งให้ดูใต้รูป)
- `readAsBytes()` ได้ `Uint8List` เก็บไว้ใน state ทั้งแสดงผล (`Image.memory`) และส่งต่อโดยไม่ต้องอ่านไฟล์ซ้ำ
- `file.mimeType` บน Android บางครั้งเป็น null จึงมี `_guessMime` เดาจากนามสกุล — Gemini ต้องการ mime_type ที่ถูกต้อง ไม่งั้นได้ 400

7.2 เขียน _analyze()

`lib/screens/photo_screen.dart` เฉลย

```dart
/// ส่งรูปไปให้ AI วิเคราะห์
Future<void> _analyze() async {
  final bytes = _imageBytes;
  if (bytes == null) return;
  setState(() {
    _loading = true;
    _error = null;
    _result = null;
  });
  try {
    final result = await widget.ai.analyzePhoto(bytes, mimeType: _mimeType);
    if (!mounted) return;
    setState(() => _result = result);
  } on AiException catch (e) {
    if (!mounted) return;
    setState(() => _error = e.message);
  } catch (e) {
    if (!mounted) return;
    setState(() => _error = 'อ่านคำตอบของ AI ไม่ได้: $e');
  } finally {
    if (mounted) setState(() => _loading = false);
  }
}
```

โครงเดียวกับ `_ask()` เป๊ะ ต่างแค่เรียก `analyzePhoto` — ถ้าเขียน Step 6 ได้ ข้อนี้ควรใช้เวลาไม่ถึง 5 นาที

7.3 เขียน GeminiAiService.analyzePhoto()

`lib/data/ai_service.dart` เฉลย

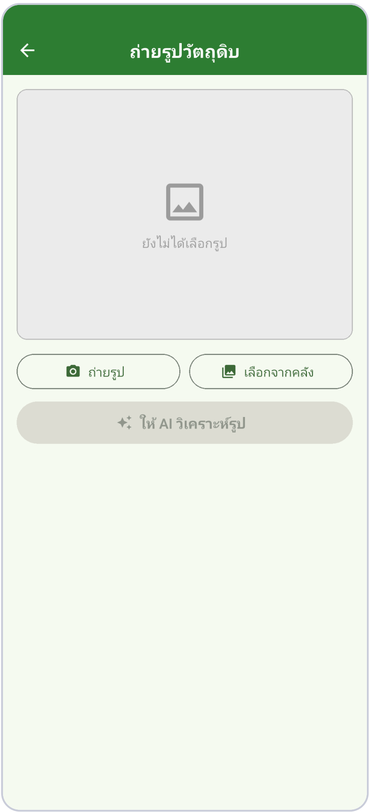
```dart
Future<PhotoAnalysis> analyzePhoto(Uint8List imageBytes, {String mimeType = 'image/jpeg'})
async {
  final text = await client.generate(
    prompt: kPhotoPrompt,
    systemInstruction: kChefSystemInstruction,
    imageBytes: imageBytes,
    imageMimeType: mimeType,
    responseSchema: kPhotoResponseSchema,
    temperature: 0.2, // งาน "ดูรูปแล้วบอก" ต้องการความแม่น ไม่ต้องการความสร้างสรรค์
  );
  return PhotoAnalysis.fromJsonText(text);
}
```

prompt และ schema สำหรับรูปอยู่ใน prompts.dart แล้ว (`kPhotoPrompt` , `kPhotoResponseSchema`) จุดต่างคือ `temperature: 0.2` — การ "บอกว่าเห็นอะไร" ต้องการความแม่น ไม่ต้องการจินตนาการ

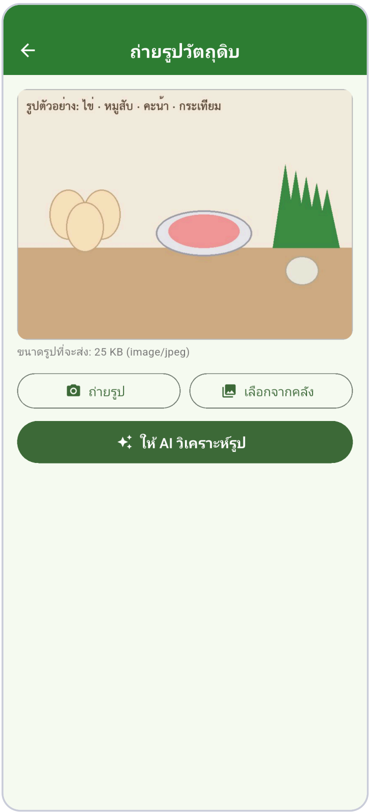
7.4 ทดสอบ

1. **บนอีมูเลเตอร์** กล้องเป็นฉากจำลอง 3 มิติ ให้ใช้ "เลือกจากคลัง" แทน: ลากไฟล์รูปอาหาร (หาจาก Google Images หรือถ่ายของในตู้เย็นแล้วส่งเข้าคอม) ไปวางบนหน้าต่างอีมูเลเตอร์ รูปจะไปอยู่ใน Downloads แล้วเลือกได้
2. **บนมือถือจริง** กด "ถ่ายรูป" ถ่ายของในครัว/โต๊ะอาหาร — สนุกกว่าและเห็นความแม่นของ AI ชัด
3. กด "ให้ AI วิเคราะห์รูป" รอ 5–15 วินาที จะได้คำบรรยาย + รายการของพร้อมปริมาณ

4. กด "แนะนำเมนูจากของ n อย่างนี้" → เปิดหน้าเมนูโดยมี chips กรอกให้แล้ว (`initialIngredients`) → กดขอเมนูต่อได้เลย
5. ลองรูปที่ไม่ใช่อาหาร (เช่น รูปโน้ตบุ๊ก) → AI ควรบอกตามจริงและรายการว่าง (กติกานี้อยู่ใน kPhotoPrompt)

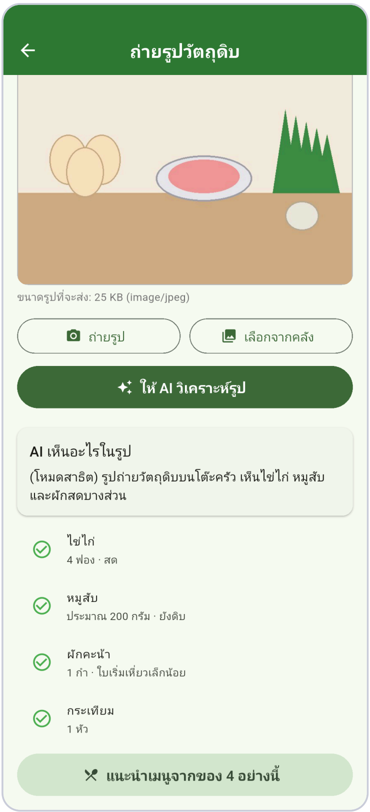

ถ่ายรูปวัตถุดิบ
ยังไม่ได้เลือกรูป
ถ่ายรูป
เลือกจากคลัง
ให้ AI วิเคราะห์รูป

ก่อนเลือกรูป


ถ่ายรูปวัตถุดิบ
รูปตัวอย่าง: ไข่ · หมูสับ · คะน้า · กระเทียม
ถ่ายรูป
เลือกจากคลัง
ให้ AI วิเคราะห์รูป

ขนาดรูปที่จะส่ง: 25 KB (image/jpeg)

เลือกรูปแล้ว (แสดงขนาด)


ถ่ายรูปวัตถุดิบ
ถ่ายรูป
เลือกจากคลัง
ให้ AI วิเคราะห์รูป
แนะนำเมนูจากของ 4 อย่างนี้

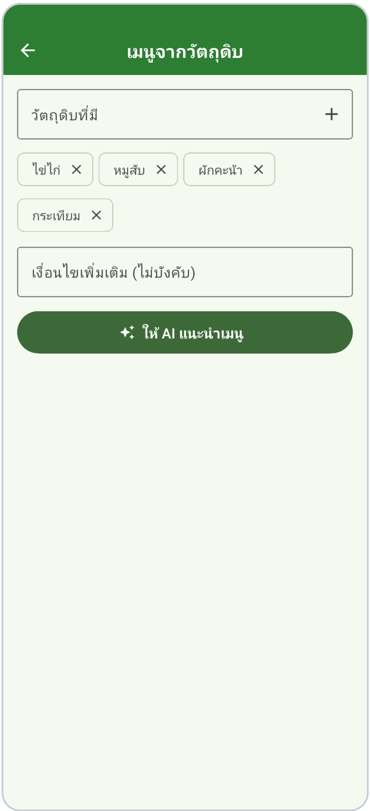
ขนาดรูปที่จะส่ง: 25 KB (image/jpeg)

AI เห็นอะไรในรูป

(โหมดสาธิต) รูปถ่ายวัตถุดิบบนโต๊ะครัว เห็นไข่ไก่ หมูสับ และผักสดบางส่วน

- ✓ ไข่ไก่ — 4 ฟอง · สด
- ✓ หมูสับ — ประมาณ 200 กรัม · ยังดิบ
- ✓ ผักคะน้า — 1 กำ · ใบเริ่มเหี่ยวเล็กน้อย
- ✓ กระเทียม — 1 หัว

ผลวิเคราะห์ (โหมดสาธิต)


เมนูจากวัตถุดิบ
วัตถุดิบที่มี
เงื่อนไขเพิ่มเติม (ไม่บังคับ)
ให้ AI แนะนำเมนู

ไข่ไก่ ✕ หมูสับ ✕ ผักคะน้า ✕ กระเทียม ✕

ส่งต่อไปหน้าเมนู

✔ Checkpoint 7

- วิเคราะห์รูปจริงได้ และรายการที่ AI เห็นตรงกับของในรูปอย่างน้อย 70%
- ส่งต่อไปหน้าเมนูแล้วได้เมนูจากของในรูป (ถ่ายภาพหน้าจอ 3 ภาพ: รูป, ผลวิเคราะห์, เมนู)
- รูปที่ไม่ใช่อาหารไม่ทำให้แอป crash

STEP 8 · อ่าน+ทำตาม

ตรวจสอบรอบสุดท้าย แก้ปัญหา และสร้าง APK

เวลาโดยประมาณ 20 นาที

เป้าหมาย: เทสต์ผ่านครบ · ไม่มีคำเตือนจาก analyzer · ได้ไฟล์ APK ติดตั้งบนมือถือ

```
flutter analyze     # ต้อง No issues found
flutter test        # ต้อง 12 passed
```

ถ้าคุณใส่บรรทัด `// ignore_for_file: ...` ที่ starter ให้มาไว้บนสุดของ 4 ไฟล์ ตอนนี้ลบออกได้แล้ว (มันมีไว้ปิดคำเตือนตอนโค้ดยังไม่ครบ)

8.1 ปัญหาที่พบบ่อย

อาการ	สาเหตุ	แก้
กดปุ่มแล้วไม่มีอะไรเกิดขึ้น / SnackBar "ยังไม่ได้เขียน"	ยังไม่ได้ลบโค้ด TODO ใน starter	ลบ SnackBar/throw UnimplementedError ออกให้หมด
"API key ไม่ถูกต้อง" ทั้งที่วางถูก	มีช่องว่าง/บรรทัดใหม่ติดมาตอน copy · key ถูกลบใน AI Studio	พิมพ์ใหม่ · สร้าง key ใหม่
429 บ่อย	ทั้งห้องใช้ Wi-Fi เดียวกัน? ไม่เกี่ยว — โควตานับต่อ key · กดถี่เกินโควตาต่อนาที	รอ 1 นาที · เปลี่ยน kGeminiModel เป็นรุ่น lite
"AI ตอบช้าเกินไป (เกิน 90 วินาที)" / รอนานมาก	โมเดล 3.x คิดนานเพราะ thinkingLevel สูง หรือ Google คนเยอะ	ตรวจว่า body มี thinkingConfig (Step 5.2) · ลด kThinkingLevel เป็น low · เปลี่ยนเป็น gemini-3.5-flash-lite · ย่อรูปให้เล็กลง
404 ไม่พบโมเดล	Google ปลดระวางชื่อโมเดลนั้นแล้ว	เปิด ai.google.dev/gemini-api/docs/models เลือกรุ่น Flash ปัจจุบัน แก้ ai_config.dart
"อ่านคำตอบของ AI ไม่ได้: FormatException"	AI ตอบ JSON ไม่ครบ (ถูกตัดที่ maxOutputTokens) หรือไม่ได้ส่ง schema	เพิ่ม maxOutputTokens · ลดจำนวนเมนู · ตรวจว่าส่ง responseSchema
หน้ารูป: 400 Invalid argument / Unsupported MIME	mime_type ไม่ตรงกับไฟล์ (เช่น HEIC จาก iPhone)	ตั้ง imageQuality ให้ image_picker แปลงเป็น JPEG หรือ map HEIC → image/heic
Unhandled Exception: setState() called after dispose()	ไม่ได้เช็ก mounted หลัง await	เพิ่ม `if (!mounted) return;` ก่อน setState ทุกจุดหลัง await

ปุ่มทดสอบขึ้น SocketException บนอีมูเลเตอร์	อีมูเลเตอร์ไม่มีเน็ต (DNS)	Cold boot อีมูเลเตอร์ หรือตั้ง DNS 8.8.8.8 ในเครื่อง
คำตอบเป็นภาษาอังกฤษ	system instruction ไม่ถูกส่ง (body ไม่มี system_instruction)	ตรวจ Step 5.2 และเทสต์ข้อแรก

8.2 สร้าง APK

```
flutter build apk --release
# ได้ไฟล์ build/app/outputs/flutter-apk/app-release.apk (~20 MB)
```

ส่งไฟล์ APK เข้ามือถือ (LINE/Drive/สาย USB) → เปิดไฟล์ → อนุญาตติดตั้งจากแหล่งที่ไม่รู้จัก → เปิดแอป → ใส่ key ที่หน้าตั้งค่า (key ไม่ได้ติดไปกับ APK — ดี เพราะถ้าแชร์ APK ให้เพื่อน เขาต้องใช้ key ของตัวเอง)

✔ Checkpoint 8

- flutter analyze สะอาด · flutter test 12/12
- APK ติดตั้งและใช้งานบนมือถือจริงได้ทั้ง 2 ฟีเจอร์

STEP 9 · ทำเอง

โจทย์ท้าทาย (เลือกทำ) และสิ่งที่ต้องส่ง

เวลาโดยประมาณ 60 นาที

เป้าหมาย: ต่อยอดแอปด้วยตัวเองอย่างน้อย 1 ข้อ และส่งงานครบ

เลือกทำ**อย่างน้อย 1 ข้อ** (ข้อ A–B ง่าย · C–D ปานกลาง · E ยาก) ทุกข้อใช้ความรู้ที่มีอยู่แล้วในแล็บนี้ มีคำใบ้ให้ แต่ไม่มีเฉลย

A · เลือกจำนวนเมนู (ง่าย)

- เพิ่ม `DropdownButton` หรือ `SegmentedButton` ให้เลือก 3 / 5 / 8 เมนู ในหน้าวัตถุดิบ
- คำใบ้: เพิ่มพารามิเตอร์ `count` ให้ `buildMenuPrompt` และ `suggestMenus` (อย่าลืมแก้ `AiService` ทั้ง 2 implementation)

B · ถามเชฟต่อ (ง่าย–ปานกลาง)

- ใต้การ์ดเมนู เพิ่มช่อง "ถามเพิ่มเกี่ยวกับเมนูนี้" เช่น "ถ้าไม่มีซอสหอยนางรมใช้อะไรแทน"
- คำใบ้: เพิ่มเมธอด `ask(String question, {String? context})` ใน AiService ที่เรียก generate **โดยไม่ส่ง** schema (ได้ข้อความธรรมดา) และใส่ชื่อเมนู+วัตถุดิบลงใน prompt เป็นบริบท
- ขั้นสูง: ทำเป็นแชตต่อเนื่อง โดยส่ง `contents` หลายรายการสลับ role `user` / `model` — ต้องแก้ `generate()` ให้รับประวัติ

C · บันทึกเมนูโปรด (ปานกลาง)

- ปุ่มหัวใจบน MenuCard → เก็บเมนูไว้ดูทีหลังโดยไม่ต้องถาม AI ใหม่ · หน้า "เมนูโปรด" บนหน้าแรก
- คำใบ้: เพิ่ม `toJson()` ใน MenuSuggestion แล้วเก็บเป็น `jsonEncode(list)` ใน shared_preferences (ดู app_settings.dart เป็นตัวอย่าง)

D · แสดงค่าใช้จ่าย token (ปานกลาง)

- แสดงใต้ผลลัพธ์ว่า "ใช้ไป 1,130 token" จาก `usageMetadata.totalTokenCount` ใน response
- คำใบ้: generate() ตอนนี้คืน String อย่างเดียว — เปลี่ยนให้คืนคลาสใหม่ `GeminiReply {String text; int totalTokens;}` แล้วไล่แก้ที่เรียกใช้ (compiler จะบอกครบทุกจุด) · เทสต์เดิมต้องยังผ่าน

E · สแกนใบเสร็จซูเปอร์ (ยาก)

- ถ่ายรูปใบเสร็จ → AI อ่านรายการสินค้า+ราคา → สรุปว่าซื้ออะไรมาบ้างและทำเมนูอะไรได้
- คำใบ้: ใช้ analyzePhoto โครงเดิม แต่เขียน prompt + schema ใหม่ (`items[{name, price, qty}]` , `total`) และหน้าจอใหม่ · ระวังรูปใบเสร็จต้องชัด ให้ maxWidth 1600

สิ่งที่ต้องส่ง

รายการ	รูปแบบ
1. ลิงก์ GitHub ของโปรเจกต์คุณ (repo ส่วนตัวที่ fork/คัดลอกไป)	ต้องมี commit อย่างน้อย 3 ครั้ง (Step 5, 6, 7) และ **ไม่มี API key ในโค้ด** — ผู้สอนจะ grep หา "AIza"
2. ภาพหน้าจอ	curl (Step 1) · ทดสอบการเชื่อมต่อ (Step 5) · เมนูจากวัตถุดิบ 2 ภาพ (Step 6) · รูป+ผลวิเคราะห์+เมนู 3 ภาพ (Step 7)
3. บันทึกการทดลอง prompt	ตาราง: แก้อะไร / คาดว่า / เห็นจริง อย่างน้อย 2 ข้อ (Step 6.5)
4. โจทย์ท้าทาย	บอกว่าทำข้อไหน + ภาพหน้าจอ + อธิบาย 3–5 บรรทัดว่าแก้ไฟล์ไหนบ้าง
5. ตอบคำถามสั้น ๆ	(ก) ทำไมแอปนี้ไม่ต้องมีเซิร์ฟเวอร์ PHP เหมือน LabStock แล้วถ้าจะทำแอปขายจริงควรมีไหม เพราะอะไร (ข) responseSchema ช่วยอะไร ถ้าไม่มีจะเกิดอะไร (ค) ถ้า Google เปลี่ยนชื่อโมเดล ต้องแก้กี่ไฟล์

เรื่อง key อีกครั้ง

- ก่อน push ขึ้น GitHub ให้ค้นหาคำว่า `AIza` ทั้งโปรเจกต์ ต้องไม่เจอ
- ถ้าเผลอ push ไปแล้ว การลบ commit ไม่พอ (ประวัติยังอยู่) — ไปลบ key ใน AI Studio แล้วสร้างใหม่ทันที

ภาคผนวก ก: ไฟล์เฉลยฉบับเต็ม (ดูหลังทำเองแล้ว)

ไฟล์ที่เหลือ (models, widgets, settings, home) เหมือนกันทั้ง starter และ done เปิดดูได้ในโปรเจกต์

lib/data/gemini_client.dart

lib/data/gemini_client.dart ไฟล์เต็ม

```dart
import 'dart:async';
import 'dart:convert';
import 'dart:io';
import 'dart:typed_data';

import 'package:http/http.dart' as http;

import 'ai_config.dart';

/// ข้อผิดพลาดจากการเรียก AI — เก็บข้อความภาษาไทยที่พร้อมแสดงให้ผู้ใช้
class AiException implements Exception {
  AiException(this.message, [this.statusCode]);
  final String message;
  final int? statusCode;

  @override
  String toString() => 'AiException($statusCode): $message';
}

/// ชั้นล่างสุด: คุยกับ Gemini ผ่าน HTTP โดยตรง (REST API `generateContent`)
///
/// ไม่รู้จัก "เมนู" หรือ "วัตถุดิบ" — รู้แค่ว่า ส่งข้อความ (+รูป) ไป แล้วได้ข้อความกลับมา
class GeminiClient {
  GeminiClient({required this.apiKey, http.Client? client, this.model = kGeminiModel})
      : _client = client ?? http.Client();

  final String apiKey;
  final String model;
  final http.Client _client;

  /// URL ปลายทาง เช่น
  /// https://generativelanguage.googleapis.com/v1beta/models/gemini-3.8-flash:generateContent
  Uri get endpoint => Uri.parse('$kGeminiBaseUrl/models/$model:generateContent');

  /// ส่งคำถามไป Gemini แล้วคืน "ข้อความตอบกลับ"
  ///
  /// - [prompt]            ข้อความที่ผู้ใช้ถาม
  /// - [systemInstruction] บทบาท/กติกาที่ให้ AI ยึด (ไม่บังคับ)
  /// - [imageBytes]        รูปภาพ (ไม่บังคับ) ส่งเป็น base64 ใน inline_data
  /// - [responseSchema]    ถ้ากำหนด จะบังคับให้ AI ตอบเป็น JSON ตามโครงสร้างนี้
  Future<String> generate({
    required String prompt,
    String? systemInstruction,
    Uint8List? imageBytes,
    String imageMimeType = 'image/jpeg',
    Map<String, dynamic>? responseSchema,
    double temperature = 0.7,
  }) async {
    if (apiKey.trim().isEmpty) {
      throw AiException('ยังไม่ได้ตั้งค่า API key — ไปที่หน้า "ตั้งค่า" ก่อน');
    }

    // 1) ประกอบ "ชิ้นส่วน" ของคำถาม: ข้อความ และรูป (ถ้ามี)
    final parts = <Map<String, dynamic>>[
      {'text': prompt},
      if (imageBytes != null)
        {
          'inline_data': {'mime_type': imageMimeType, 'data': base64Encode(imageBytes)},
        },
    ];

    // 2) ประกอบ body ตามรูปแบบที่ Gemini ต้องการ
    final body = <String, dynamic>{
      if (systemInstruction != null)
        'system_instruction': {
          'parts': [
            {'text': systemInstruction},
          ],
        },
      'contents': [
        {'role': 'user', 'parts': parts},
      ],
      'generationConfig': {
        'temperature': temperature,
        'maxOutputTokens': 8192,
        // ลดเวลาคิดของโมเดล (ไม่ใส่ = medium ซึ่งช้ากว่ามาก)
        if (kThinkingLevel.isNotEmpty) 'thinkingConfig': {'thinkingLevel': kThinkingLeve
l},
        if (responseSchema != null) 'responseMimeType': 'application/json',
        if (responseSchema != null) 'responseSchema': responseSchema,
      },
    };

    // 3) ยิง HTTP POST — key ส่งใน header ไม่ใส่ใน URL
    http.Response res;
    try {
      res = await _client
          .post(
            endpoint,
            headers: {'Content-Type': 'application/json', 'x-goog-api-key': apiKey},
            body: jsonEncode(body),
          )
          .timeout(kAiTimeout);
    } on TimeoutException {
      throw AiException('AI ตอบช้าเกินไป (เกิน ${kAiTimeout.inSeconds} วินาที) ลองใหม่อีกครั้ง');
    } on SocketException {
      throw AiException('เชื่อมต่ออินเทอร์เน็ตไม่ได้ ตรวจสอบ Wi-Fi หรือเน็ตมือถือ');
    } on http.ClientException catch (e) {
      throw AiException('ส่งคำขอไม่สำเร็จ: ${e.message}');
    }

    // 4) ตรวจสถานะ HTTP
    if (res.statusCode != 200) {
      throw AiException(describeHttpError(res.statusCode, res.body), res.statusCode);
    }

    // 5) แกะข้อความออกจาก JSON ที่ตอบกลับ
    return extractText(jsonDecode(res.body) as Map<String, dynamic>);
  }

  /// แปลงรหัสสถานะ HTTP + body เป็นข้อความภาษาไทย
  static String describeHttpError(int status, String body) {
    String serverMessage = '';
    try {
      final json = jsonDecode(body) as Map<String, dynamic>;
      serverMessage = (json['error']?['message'] ?? '').toString();
    } catch (_) {}
    final detail = serverMessage.isEmpty ? '' : '\n($serverMessage)';
    switch (status) {
      case 400:
        if (serverMessage.contains('API key')) {
          return 'API key ไม่ถูกต้อง ตรวจสอบที่หน้า "ตั้งค่า"$detail';
        }
        return 'คำขอผิดรูปแบบ (400)$detail';
      case 401:
      case 403:
        return 'API key นี้ไม่มีสิทธิ์ใช้งาน ($status)$detail';
      case 404:
        return 'ไม่พบโมเดล "$kGeminiModel" — ชื่อโมเดลอาจเปลี่ยน ตรวจสอบใน ai_config.dart$detail';
      case 429:
        return 'ใช้โควตาฟรีเกินกำหนดชั่วคราว (429) รอสัก 1 นาทีแล้วลองใหม่$detail';
      case 500:
      case 503:
        return 'เซิร์ฟเวอร์ Gemini ขัดข้องชั่วคราว ($status) ลองใหม่อีกครั้ง$detail';
      default:
        return 'เกิดข้อผิดพลาด HTTP $status$detail';
    }
  }

  /// ดึงข้อความจาก JSON ตอบกลับของ Gemini
  ///
  /// รูปแบบ: { "candidates": [ { "content": { "parts": [ { "text": "..." } ] }, "finishReason": "STOP" } ] }
  static String extractText(Map<String, dynamic> json) {
    final feedback = json['promptFeedback'] as Map<String, dynamic>?;
    if (feedback?['blockReason'] != null) {
      throw AiException('AI ปฏิเสธคำถามนี้ (${feedback!['blockReason']}) ลองเปลี่ยนคำถาม');
    }
    final candidates = json['candidates'] as List<dynamic>?;
    if (candidates == null || candidates.isEmpty) {
      throw AiException('AI ไม่ได้ส่งคำตอบกลับมา');
    }
    final first = candidates.first as Map<String, dynamic>;
    final parts = (first['content']?['parts'] as List<dynamic>?) ?? const [];
    final text = parts
        .map((p) => (p as Map<String, dynamic>)['text'])
        .whereType<String>()
        .join();
    if (text.trim().isEmpty) {
      final reason = first['finishReason'] ?? 'ไม่ทราบสาเหตุ';
      throw AiException('AI ตอบกลับมาว่างเปล่า (finishReason: $reason)');
    }
    return text;
  }
}
```

lib/data/ai_service.dart

`lib/data/ai_service.dart` ไฟล์เต็ม

```dart
import 'dart:typed_data';

import '../models/menu_suggestion.dart';
import '../models/photo_analysis.dart';
import 'gemini_client.dart';
import 'prompts.dart';

/// "งาน" ที่แอปเราต้องการจาก AI — หน้าจอต่าง ๆ เรียกผ่าน interface นี้
///
/// มี 2 แบบ: [GeminiAiService] ใช้ AI จริง และ [DemoAiService] ตอบข้อมูลจำลอง
/// (ไว้ดู UI โดยไม่ต้องมี API key และใช้ในเทสต์)
abstract class AiService {
  /// ส่งรายการวัตถุดิบ → ได้เมนูแนะนำ
  Future<MenuResult> suggestMenus(List<String> ingredients, {String? extra});

  /// ส่งรูป → ได้รายการของที่เห็นในรูป
  Future<PhotoAnalysis> analyzePhoto(Uint8List imageBytes, {String mimeType = 'image/jpeg'});

  /// ทดสอบว่า key ใช้ได้ไหม → คืนข้อความสั้น ๆ จาก AI
  Future<String> ping();
}

/// ใช้ Gemini จริง — ประกอบ prompt + schema แล้วแปลงคำตอบเป็น model ของเรา
class GeminiAiService implements AiService {
  GeminiAiService(this.client);
  final GeminiClient client;

  @override
  Future<MenuResult> suggestMenus(List<String> ingredients, {String? extra}) async {
    final text = await client.generate(
      prompt: buildMenuPrompt(ingredients, extra: extra),
      systemInstruction: kChefSystemInstruction,
      responseSchema: kMenuResponseSchema,
    );
    return MenuResult.fromJsonText(text);
  }

  @override
  Future<PhotoAnalysis> analyzePhoto(Uint8List imageBytes, {String mimeType = 'image/jpeg'}) async {
    final text = await client.generate(
      prompt: kPhotoPrompt,
      systemInstruction: kChefSystemInstruction,
      imageBytes: imageBytes,
      imageMimeType: mimeType,
      responseSchema: kPhotoResponseSchema,
      temperature: 0.2, // งาน "ดูรูปแล้วบอก" ต้องการความแม่น ไม่ต้องการความสร้างสรรค์
    );
    return PhotoAnalysis.fromJsonText(text);
  }

  @override
  Future<String> ping() => client.generate(
        prompt: 'ทักทายสั้น ๆ 1 ประโยค แล้วบอกชื่อโมเดลของคุณ',
        temperature: 0.2,
      );
}

/// โหมดสาธิต — ตอบข้อมูลจำลองหลังหน่วงเวลาสั้น ๆ ไม่ต้องต่อเน็ต ไม่ใช้ key
```

```
class DemoAiService implements AiService {
  DemoAiService({this.delay = const Duration(milliseconds: 800)});
  final Duration delay;

  @override
  Future<MenuResult> suggestMenus(List<String> ingredients, {String? extra}) async {
    await Future<void>.delayed(delay);
    final have = ingredients.isEmpty ? 'ของในครัว' : ingredients.join(', ');
    return MenuResult(
      summary: '(โหมดสาธิต) จาก $have ทำเมนูง่าย ๆ ได้หลายอย่าง นี่คือตัวอย่าง',
      menus: [
        MenuSuggestion(
          name: 'ไข่เจียวหมูสับ',
          description: 'เมนูพื้นฐานที่ใช้ไข่กับหมูสับ ทำไว ทานกับข้าวสวยร้อน ๆ',
          ingredients: ['ไข่ไก่ 2 ฟอง', 'หมูสับ 50 กรัม', 'น้ำปลา 1 ช้อนชา', 'น้ำมันพืช 3 ช้อนโต๊ะ'],
          missing: [],
          steps: [
            'ตีไข่กับหมูสับและน้ำปลาให้เข้ากัน',
            'ตั้งกระทะ ใส่น้ำมัน รอให้ร้อนจัด',
            'เทไข่ลงไป ทอดจนเหลืองกรอบทั้งสองด้าน',
            'ตักขึ้นเสิร์ฟกับข้าวสวยและซอสพริก',
          ],
          time: '10 นาที',
          difficulty: 'ง่าย',
        ),
        MenuSuggestion(
          name: 'ผัดกะเพราหมูสับ',
          description: 'เมนูสิ้นคิดยอดนิยม หอมกะเพรา เผ็ดกำลังดี',
          ingredients: ['หมูสับ 150 กรัม', 'กระเทียม 3 กลีบ', 'พริกขี้หนู 5 เม็ด', 'ใบกะเพรา 1 กำมือ',
'ซอสหอยนางรม 1 ช้อนโต๊ะ'],
          missing: ['ใบกะเพรา', 'พริกขี้หนู'],
          steps: [
            'โขลกกระเทียมกับพริกพอหยาบ',
            'ตั้งกระทะ ผัดกระเทียมพริกกับน้ำมันให้หอม',
            'ใส่หมูสับ ผัดจนสุก ปรุงรสด้วยซอสหอยนางรม น้ำปลา น้ำตาลเล็กน้อย',
            'ใส่ใบกะเพรา ผัดเร็ว ๆ แล้วปิดไฟ',
          ],
          time: '15 นาที',
          difficulty: 'ง่าย',
        ),
        MenuSuggestion(
          name: 'ต้มจืดไข่น้ำ',
          description: 'ซุปใสรสอ่อน ทำง่าย เหมาะกับคนไม่กินเผ็ด',
          ingredients: ['ไข่ไก่ 2 ฟอง', 'หมูสับ 50 กรัม', 'น้ำซุป 2 ถ้วย', 'ต้นหอม 1 ต้น'],
          missing: ['ต้นหอม'],
          steps: [
            'ต้มน้ำซุปให้เดือด ใส่หมูสับปั้นเป็นก้อนเล็ก ๆ',
            'ตีไข่ แล้วค่อย ๆ เทลงในน้ำเดือด คนเบา ๆ ให้ไข่เป็นเส้น',
            'ปรุงรสด้วยน้ำปลา โรยต้นหอม พริกไทย',
          ],
          time: '15 นาที',
          difficulty: 'ปานกลาง',
        ),
      ],
    );
  }

  @override
  Future<PhotoAnalysis> analyzePhoto(Uint8List imageBytes, {String mimeType = 'image/jpe
g'}) async {
    await Future<void>.delayed(delay);
    return PhotoAnalysis(
      description: '(โหมดสาธิต) รูปถ่ายวัตถุดิบบนโต๊ะครัว เห็นไข่ไก่ หมูสับ และผักสดบางส่วน',
      items: [
        FoundItem(name: 'ไข่ไก่', amount: '4 ฟอง', note: 'สด'),
        FoundItem(name: 'หมูสับ', amount: 'ประมาณ 200 กรัม', note: 'ยังดิบ'),
        FoundItem(name: 'ผักคะน้า', amount: '1 กำ', note: 'ใบเริ่มเหี่ยวเล็กน้อย'),
        FoundItem(name: 'กระเทียม', amount: '1 หัว', note: ''),
      ],
    );
  }

  @override
  Future<String> ping() async {
    await Future<void>.delayed(delay);
    return 'สวัสดีครับ นี่คือโหมดสาธิต ยังไม่ได้ต่อกับ Gemini จริง';
  }
}
```

lib/screens/ingredients_screen.dart

lib/screens/ingredients_screen.dart ไฟล์เต็ม

```
import 'package:flutter/material.dart';

import '../data/ai_service.dart';
import '../data/gemini_client.dart';
import '../models/menu_suggestion.dart';
import '../widgets/error_box.dart';
import '../widgets/waiting_indicator.dart';
import '../widgets/menu_card.dart';

/// หน้า "พิมพ์วัตถุดิบ" → ส่งให้ AI → แสดงเมนูแนะนำ
class IngredientsScreen extends StatefulWidget {
  const IngredientsScreen({super.key, required this.ai, this.initialIngredients = const []});

  final AiService ai;

  /// รายการเริ่มต้น (หน้า "ถ่ายรูป" ส่งมาให้)
  final List<String> initialIngredients;

  @override
  State<IngredientsScreen> createState() => _IngredientsScreenState();
}

class _IngredientsScreenState extends State<IngredientsScreen> {
  final _inputCtrl = TextEditingController();
  final _extraCtrl = TextEditingController();
  late final List<String> _ingredients = [...widget.initialIngredients];

  bool _loading = false;
  String? _error;
  MenuResult? _result;

  @override
  void dispose() {
    _inputCtrl.dispose();
    _extraCtrl.dispose();
    super.dispose();
  }

  /// เพิ่มวัตถุดิบจากช่องพิมพ์ (พิมพ์หลายอย่างคั่นด้วย , ได้)
  void _addIngredient() {
    final items = _inputCtrl.text
        .split(RegExp(r'[,\n]'))
        .map((e) => e.trim())
        .where((e) => e.isNotEmpty && !_ingredients.contains(e));
    setState(() => _ingredients.addAll(items));
    _inputCtrl.clear();
  }

  /// ส่งรายการวัตถุดิบไปถาม AI
  Future<void> _ask() async {
    if (_ingredients.isEmpty) {
      ScaffoldMessenger.of(context)
          .showSnackBar(const SnackBar(content: Text('ใส่วัตถุดิบอย่างน้อย 1 อย่างก่อน')));
      return;
    }
    setState(() {
      _loading = true;
      _error = null;
      _result = null;
    });
    try {
      final result = await widget.ai.suggestMenus(_ingredients, extra: _extraCtrl.text);
      if (!mounted) return;
      setState(() => _result = result);
    } on AiException catch (e) {
      if (!mounted) return;
      setState(() => _error = e.message);
    } catch (e) {
      // เช่น AI ตอบ JSON ผิดรูปแบบ → FormatException
      if (!mounted) return;
      setState(() => _error = 'อ่านคำตอบของ AI ไม่ได้: $e');
    } finally {
      if (mounted) setState(() => _loading = false);
    }
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('เมนูจากวัตถุดิบ')),
      body: ListView(
        padding: const EdgeInsets.all(16),
        children: [
          // ---- ช่องพิมพ์วัตถุดิบ ----
          TextField(
            controller: _inputCtrl,
            decoration: InputDecoration(
              labelText: 'วัตถุดิบที่มี',
              hintText: 'เช่น ไข่, หมูสับ, กระเทียม',
              suffixIcon: IconButton(icon: const Icon(Icons.add), onPressed: _addIngredient),
            ),
            textInputAction: TextInputAction.done,
            onSubmitted: (_) => _addIngredient(),
          ),
          const SizedBox(height: 10),
          // ---- รายการที่เพิ่มแล้ว (chips) ----
          if (_ingredients.isEmpty)
            Text('ยังไม่มีรายการ — พิมพ์แล้วกด + หรือ Enter',
                style: TextStyle(color: Colors.grey.shade600))
          else
            Wrap(
              spacing: 6,
              runSpacing: 4,
              children: _ingredients
                  .map((item) => InputChip(
                        label: Text(item),
                        onDeleted: () => setState(() => _ingredients.remove(item)),
                      ))
                  .toList(),
            ),
          const SizedBox(height: 12),
          TextField(
            controller: _extraCtrl,
            decoration: const InputDecoration(
              labelText: 'เงื่อนไขเพิ่มเติม (ไม่บังคับ)',
              hintText: 'เช่น ไม่กินเผ็ด, ทำใน 15 นาที, มังสวิรัติ',
            ),
          ),
          const SizedBox(height: 16),
          FilledButton.icon(
            onPressed: _loading ? null : _ask,
            icon: const Icon(Icons.auto_awesome),
            label: Text(_loading ? 'AI กำลังคิด...' : 'ให้ AI แนะนำเมนู'),
          ),
          const SizedBox(height: 16),
          // ---- ส่วนแสดงผล: กำลังโหลด / ผิดพลาด / ผลลัพธ์ ----
          if (_loading) const WaitingIndicator(),
          if (_error != null) ErrorBox(message: _error!, onRetry: _ask),
          if (_result != null) ..._buildResult(_result!),
        ],
      ),
    );
  }

  List<Widget> _buildResult(MenuResult r) => [
        if (r.summary.isNotEmpty)
          Padding(
            padding: const EdgeInsets.only(bottom: 8),
            child: Text(r.summary, style: const TextStyle(fontStyle: FontStyle.italic)),
          ),
        Text('เมนูแนะนำ ${r.menus.length} รายการ',
            style: Theme.of(context).textTheme.titleMedium),
        ...r.menus.asMap().entries.map((e) => MenuCard(menu: e.value, initiallyExpanded:
e.key == 0)),
      ];
}
```

lib/screens/photo_screen.dart

lib/screens/photo_screen.dart ไฟล์เต็ม

```dart
import 'dart:typed_data';

import 'package:flutter/material.dart';
import 'package:image_picker/image_picker.dart';

import '../data/ai_service.dart';
import '../data/gemini_client.dart';
import '../models/photo_analysis.dart';
import '../widgets/error_box.dart';
import '../widgets/waiting_indicator.dart';
import 'ingredients_screen.dart';

/// หน้า "ถ่ายรูป" → ส่งรูปให้ AI → แสดงของที่เห็น → ต่อยอดเป็นเมนู
class PhotoScreen extends StatefulWidget {
  const PhotoScreen({super.key, required this.ai});
  final AiService ai;

  @override
  State<PhotoScreen> createState() => _PhotoScreenState();
}

class _PhotoScreenState extends State<PhotoScreen> {
  final _picker = ImagePicker();

  Uint8List? _imageBytes; // รูปที่เลือก (เป็น bytes พร้อมส่ง)
  String _mimeType = 'image/jpeg';
  bool _loading = false;
  String? _error;
  PhotoAnalysis? _result;

  /// เปิดกล้องหรือคลังรูป แล้วย่อรูปให้เล็กลงก่อนส่ง (ประหยัดเน็ตและโควตา)
  Future<void> _pickImage(ImageSource source) async {
    final XFile? file = await _picker.pickImage(
      source: source,
      maxWidth: 1024,
      maxHeight: 1024,
      imageQuality: 80,
    );
    if (file == null) return; // ผู้ใช้กดยกเลิก
    final bytes = await file.readAsBytes();
    setState(() {
      _imageBytes = bytes;
      _mimeType = file.mimeType ?? _guessMime(file.name);
      _result = null;
      _error = null;
    });
  }

  static String _guessMime(String name) {
    final n = name.toLowerCase();
    if (n.endsWith('.png')) return 'image/png';
    if (n.endsWith('.webp')) return 'image/webp';
    if (n.endsWith('.heic')) return 'image/heic';
    return 'image/jpeg';
  }

  /// ส่งรูปไปให้ AI วิเคราะห์
  Future<void> _analyze() async {
    final bytes = _imageBytes;
    if (bytes == null) return;
    setState(() {
      _loading = true;
      _error = null;
      _result = null;
    });
    try {
      final result = await widget.ai.analyzePhoto(bytes, mimeType: _mimeType);
      if (!mounted) return;
      setState(() => _result = result);
    } on AiException catch (e) {
      if (!mounted) return;
      setState(() => _error = e.message);
    } catch (e) {
      if (!mounted) return;
      setState(() => _error = 'อ่านคำตอบของ AI ไม่ได้: $e');
    } finally {
      if (mounted) setState(() => _loading = false);
    }
  }

  /// เอาชื่อของที่ AI เห็น ไปเปิดหน้า "เมนูจากวัตถุดิบ"
  void _suggestMenus() {
    final names = _result?.itemNames ?? const [];
    Navigator.push(
      context,
      MaterialPageRoute(builder: (_) => IngredientsScreen(ai: widget.ai, initialIngredient
s: names)),
    );
  }

  @override
  Widget build(BuildContext context) {
    final sizeKb = _imageBytes == null ? null : (_imageBytes!.lengthInBytes / 1024).round
();
    return Scaffold(
      appBar: AppBar(title: const Text('ถ่ายรูปวัตถุดิบ')),
      body: ListView(
        padding: const EdgeInsets.all(16),
        children: [
          // ---- พื้นที่แสดงรูป ----
          AspectRatio(
            aspectRatio: 4 / 3,
            child: Container(
              decoration: BoxDecoration(
                color: Colors.grey.shade200,
                borderRadius: BorderRadius.circular(12),
                border: Border.all(color: Colors.grey.shade400),
              ),
              clipBehavior: Clip.antiAlias,
              child: _imageBytes == null
                  ? const Center(
                      child: Column(
                        mainAxisSize: MainAxisSize.min,
                        children: [
                          Icon(Icons.image_outlined, size: 56, color: Colors.grey),
                          SizedBox(height: 8),
                          Text('ยังไม่ได้เลือกรูป', style: TextStyle(color: Colors.grey)),
                        ],
                      ),
                    )
                  : Image.memory(_imageBytes!, fit: BoxFit.cover),
            ),
          ),
          if (sizeKb != null)
            Padding(
              padding: const EdgeInsets.only(top: 4),
              child: Text('ขนาดรูปที่จะส่ง: $sizeKb KB ($_mimeType)',
                  style: TextStyle(fontSize: 12, color: Colors.grey.shade600)),
            ),
          const SizedBox(height: 12),
          Row(
            children: [
              Expanded(
                child: OutlinedButton.icon(
                  onPressed: _loading ? null : () => _pickImage(ImageSource.camera),
                  icon: const Icon(Icons.photo_camera),
                  label: const Text('ถ่ายรูป'),
                ),
              ),
              const SizedBox(width: 10),
              Expanded(
                child: OutlinedButton.icon(
                  onPressed: _loading ? null : () => _pickImage(ImageSource.gallery),
                  icon: const Icon(Icons.photo_library),
                  label: const Text('เลือกจากคลัง'),
                ),
              ),
            ],
          ),
          const SizedBox(height: 10),
          FilledButton.icon(
            onPressed: (_imageBytes == null || _loading) ? null : _analyze,
            icon: const Icon(Icons.auto_awesome),
            label: Text(_loading ? 'AI กำลังดูรูป...' : 'ให้ AI วิเคราะห์รูป'),
          ),
          const SizedBox(height: 16),
          if (_loading) const WaitingIndicator(),
          if (_error != null) ErrorBox(message: _error!, onRetry: _analyze),
          if (_result != null) ..._buildResult(_result!),
        ],
      ),
    );
  }

  List<Widget> _buildResult(PhotoAnalysis r) => [
        Card(
          child: Padding(
            padding: const EdgeInsets.all(14),
            child: Column(
              crossAxisAlignment: CrossAxisAlignment.start,
              children: [
                Text('AI เห็นอะไรในรูป', style: Theme.of(context).textTheme.titleMedium),
                const SizedBox(height: 6),
                Text(r.description),
              ],
            ),
          ),
        ),
        if (r.items.isEmpty)
          const Padding(padding: EdgeInsets.all(8), child: Text('ไม่พบวัตถุดิบในรูปนี้'))
        else
          ...r.items.map(
            (it) => ListTile(
              dense: true,
              leading: const Icon(Icons.check_circle_outline, color: Colors.green),
              title: Text(it.name),
              subtitle: Text([it.amount, it.note].where((s) => s.isNotEmpty).join(' · ')),
            ),
          ),
        if (r.items.isNotEmpty) ...[
          const SizedBox(height: 8),
          FilledButton.tonalIcon(
            onPressed: _suggestMenus,
            icon: const Icon(Icons.restaurant_menu),
            label: Text('แนะนำเมนูจากของ ${r.items.length} อย่างนี้'),
          ),
        ],
      ];
}
```

ภาคผนวก ข: starter → done เปลี่ยนอะไรบ้าง (diff)

lib/data/gemini_client.dart starter → done · +70 / -15 บรรทัด

```diff
@@ -1,4 +1,2 @@
-// ignore_for_file: unused_import, unused_field, unused_element, prefer_final_fields, unused_local_variable
-// (คำเตือนพวกนี้จะหายไปเองเมื่อเขียน TODO ครบ — ลบบรรทัดนี้ได้เมื่อทำเสร็จ)
 import 'dart:async';
 import 'dart:convert';
@@ -53,17 +51,59 @@ class GeminiClient {
     }

-    // TODO(Step 5.1): ประกอบ parts = [ {'text': prompt}, (ถ้ามีรูป) {'inline_data': {...}} ]
+    // 1) ประกอบ "ชิ้นส่วน" ของคำถาม: ข้อความ และรูป (ถ้ามี)
+    final parts = <Map<String, dynamic>>[
+      {'text': prompt},
+      if (imageBytes != null)
+        {
+          'inline_data': {'mime_type': imageMimeType, 'data': base64Encode(imageBytes)},
+        },
+    ];

-    // TODO(Step 5.2): ประกอบ body ตามรูปแบบของ Gemini
-    //   system_instruction (ถ้ามี) · contents · generationConfig (temperature, maxOutputTokens,
-    //   thinkingConfig.thinkingLevel = kThinkingLevel, และถ้ามี responseSchema ให้ใส่ responseMimeType + responseSchema)
+    // 2) ประกอบ body ตามรูปแบบที่ Gemini ต้องการ
+    final body = <String, dynamic>{
+      if (systemInstruction != null)
+        'system_instruction': {
+          'parts': [
+            {'text': systemInstruction},
+          ],
+        },
+      'contents': [
+        {'role': 'user', 'parts': parts},
+      ],
+      'generationConfig': {
+        'temperature': temperature,
+        'maxOutputTokens': 8192,
+        // ลดเวลาคิดของโมเดล (ไม่ใส่ = medium ซึ่งช้ากว่ามาก)
+        if (kThinkingLevel.isNotEmpty) 'thinkingConfig': {'thinkingLevel': kThinkingLevel},
+        if (responseSchema != null) 'responseMimeType': 'application/json',
+        if (responseSchema != null) 'responseSchema': responseSchema,
+      },
+    };

-    // TODO(Step 5.3): ยิง HTTP POST ไปที่ endpoint พร้อม header 'x-goog-api-key'
-    //   ครอบด้วย try/catch: TimeoutException, SocketException, http.ClientException → AiException
+    // 3) ยิง HTTP POST — key ส่งใน header ไม่ใส่ใน URL
+    http.Response res;
+    try {
+      res = await _client
+          .post(
+            endpoint,
+            headers: {'Content-Type': 'application/json', 'x-goog-api-key': apiKey},
+            body: jsonEncode(body),
+          )
+          .timeout(kAiTimeout);
+    } on TimeoutException {
```

```
+      throw AiException('AI ตอบช้าเกินไป (เกิน ${kAiTimeout.inSeconds} วินาที) ลองใหม่อีกครั้ง');
+    } on SocketException {
+      throw AiException('เชื่อมต่ออินเทอร์เน็ตไม่ได้ ตรวจสอบ Wi-Fi หรือเน็ตมือถือ');
+    } on http.ClientException catch (e) {
+      throw AiException('ส่งคำขอไม่สำเร็จ: ${e.message}');
+    }

-    // TODO(Step 5.4): ถ้า statusCode != 200 → throw AiException(describeHttpError(...), s
tatusCode)
+    // 4) ตรวจสถานะ HTTP
+    if (res.statusCode != 200) {
+      throw AiException(describeHttpError(res.statusCode, res.body), res.statusCode);
+    }

-    // TODO(Step 5.5): แกะข้อความด้วย extractText(jsonDecode(res.body))
-    throw UnimplementedError('ยังไม่ได้เขียน generate() — ทำตามคู่มือ Step 5');
+    // 5) แกะข้อความออกจาก JSON ที่ตอบกลับ
+    return extractText(jsonDecode(res.body) as Map<String, dynamic>);
   }

@@ -101,8 +141,23 @@ class GeminiClient {
   /// รูปแบบ: { "candidates": [ { "content": { "parts": [ { "text": "..." } ] }, "finishRea
son": "STOP" } ] }
   static String extractText(Map<String, dynamic> json) {
-    // TODO(Step 5.6): ดึง candidates[0].content.parts[*].text มาต่อกัน
-    //   - ถ้า promptFeedback.blockReason มีค่า → throw AiException('AI ปฏิเสธคำถามนี้ ...')
-    //   - ถ้า candidates ว่าง หรือข้อความว่าง → throw AiException พร้อม finishReason
-    throw UnimplementedError('ยังไม่ได้เขียน extractText() — ทำตามคู่มือ Step 5');
+    final feedback = json['promptFeedback'] as Map<String, dynamic>?;
+    if (feedback?['blockReason'] != null) {
+      throw AiException('AI ปฏิเสธคำถามนี้ (${feedback!['blockReason']}) ลองเปลี่ยนคำถาม');
+    }
+    final candidates = json['candidates'] as List<dynamic>?;
+    if (candidates == null || candidates.isEmpty) {
+      throw AiException('AI ไม่ได้ส่งคำตอบกลับมา');
+    }
+    final first = candidates.first as Map<String, dynamic>;
+    final parts = (first['content']?['parts'] as List<dynamic>?) ?? const [];
+    final text = parts
+        .map((p) => (p as Map<String, dynamic>)['text'])
+        .whereType<String>()
+        .join();
+    if (text.trim().isEmpty) {
+      final reason = first['finishReason'] ?? 'ไม่ทราบสาเหตุ';
+      throw AiException('AI ตอบกลับมาว่างเปล่า (finishReason: $reason)');
+    }
+    return text;
   }
 }
```

`lib/data/ai_service.dart` starter → done · +15 / -8 บรรทัด

```diff
@@ -1,4 +1,2 @@
-// ignore_for_file: unused_import, unused_field, unused_element, prefer_final_fields, unused_local_variable
-// (คำเตือนพวกนี้จะหายไปเองเมื่อเขียน TODO ครบ — ลบบรรทัดนี้ได้เมื่อทำเสร็จ)
 import 'dart:typed_data';

@@ -30,14 +28,23 @@ class GeminiAiService implements AiService {
   @override
   Future<MenuResult> suggestMenus(List<String> ingredients, {String? extra}) async {
-    // TODO(Step 6.2): เรียก client.generate(prompt: buildMenuPrompt(...), systemInstruction: ...,
-    //   responseSchema: kMenuResponseSchema) แล้วแปลงด้วย MenuResult.fromJsonText(...)
-    throw UnimplementedError('ยังไม่ได้เขียน suggestMenus() — ทำตามคู่มือ Step 6');
+    final text = await client.generate(
+      prompt: buildMenuPrompt(ingredients, extra: extra),
+      systemInstruction: kChefSystemInstruction,
+      responseSchema: kMenuResponseSchema,
+    );
+    return MenuResult.fromJsonText(text);
   }

   @override
   Future<PhotoAnalysis> analyzePhoto(Uint8List imageBytes, {String mimeType = 'image/jpeg'}) async {
-    // TODO(Step 7.3): เหมือน suggestMenus แต่ส่ง imageBytes + imageMimeType และใช้ kPhotoPrompt,
-    //   kPhotoResponseSchema, temperature ต่ำ (0.2) แล้วแปลงด้วย PhotoAnalysis.fromJsonText(...)
-    throw UnimplementedError('ยังไม่ได้เขียน analyzePhoto() — ทำตามคู่มือ Step 7');
+    final text = await client.generate(
+      prompt: kPhotoPrompt,
+      systemInstruction: kChefSystemInstruction,
+      imageBytes: imageBytes,
+      imageMimeType: mimeType,
+      responseSchema: kPhotoResponseSchema,
+      temperature: 0.2, // งาน "ดูรูปแล้วบอก" ต้องการความแม่น ไม่ต้องการความสร้างสรรค์
+    );
+    return PhotoAnalysis.fromJsonText(text);
   }
```

lib/screens/ingredients_screen.dart starter → done · +19 / −9 บรรทัด

```diff
@@ -1,4 +1,2 @@
-// ignore_for_file: unused_import, unused_field, unused_element, prefer_final_fields, unused_local_variable
-// (คำเตือนพวกนี้จะหายไปเองเมื่อเขียน TODO ครบ — ลบบรรทัดนี้ได้เมื่อทำเสร็จ)
 import 'package:flutter/material.dart';

@@ -56,11 +54,23 @@ class _IngredientsScreenState extends State<IngredientsScreen> {
       return;
     }
-    // TODO(Step 6.3): 1) setState เริ่มโหลด ล้าง error/result เก่า
-    //   2) await widget.ai.suggestMenus(_ingredients, extra: _extraCtrl.text)
-    //   3) on AiException → _error = e.message · catch อื่น ๆ → 'อ่านคำตอบของ AI ไม่ได้'
-    //   4) finally → _loading = false (เช็ก mounted ก่อน setState ทุกครั้ง)
-    ScaffoldMessenger.of(context).showSnackBar(
-      const SnackBar(content: Text('ยังไม่ได้เขียน _ask() — ทำตามคู่มือ Step 6')),
-    );
+    setState(() {
+      _loading = true;
+      _error = null;
+      _result = null;
+    });
+    try {
+      final result = await widget.ai.suggestMenus(_ingredients, extra: _extraCtrl.text);
+      if (!mounted) return;
+      setState(() => _result = result);
+    } on AiException catch (e) {
+      if (!mounted) return;
+      setState(() => _error = e.message);
+    } catch (e) {
+      // เช่น AI ตอบ JSON ผิดรูปแบบ → FormatException
+      if (!mounted) return;
+      setState(() => _error = 'อ่านคำตอบของ AI ไม่ได้: $e');
+    } finally {
+      if (mounted) setState(() => _loading = false);
+    }
   }
```

lib/screens/photo_screen.dart starter → done · +31 / −13 บรรทัด

```diff
@@ -1,4 +1,2 @@
-// ignore_for_file: unused_import, unused_field, unused_element, prefer_final_fields, unused_local_variable
-// (คำเตือนพวกนี้จะหายไปเองเมื่อเขียน TODO ครบ — ลบบรรทัดนี้ได้เมื่อทำเสร็จ)
 import 'dart:typed_data';

@@ -33,11 +31,18 @@ class _PhotoScreenState extends State<PhotoScreen> {
   /// เปิดกล้องหรือคลังรูป แล้วย่อรูปให้เล็กลงก่อนส่ง (ประหยัดเน็ตและโควตา)
   Future<void> _pickImage(ImageSource source) async {
-    // TODO(Step 7.1): เปิดกล้อง/คลังด้วย _picker.pickImage(source: source, maxWidth: 1024,
-    //   maxHeight: 1024, imageQuality: 80) → ถ้า null (ผู้ใช้ยกเลิก) ให้ return
-    //   อ่าน bytes ด้วย file.readAsBytes() แล้ว setState: _imageBytes, _mimeType
-    //   (file.mimeType ?? _guessMime(file.name)) และล้าง _result/_error
-    ScaffoldMessenger.of(context).showSnackBar(
-      const SnackBar(content: Text('ยังไม่ได้เขียน _pickImage() — ทำตามคู่มือ Step 7')),
+    final XFile? file = await _picker.pickImage(
+      source: source,
+      maxWidth: 1024,
+      maxHeight: 1024,
+      imageQuality: 80,
     );
+    if (file == null) return; // ผู้ใช้กดยกเลิก
+    final bytes = await file.readAsBytes();
+    setState(() {
+      _imageBytes = bytes;
+      _mimeType = file.mimeType ?? _guessMime(file.name);
+      _result = null;
+      _error = null;
+    });
   }

@@ -54,9 +59,22 @@ class _PhotoScreenState extends State<PhotoScreen> {
     final bytes = _imageBytes;
     if (bytes == null) return;
-    // TODO(Step 7.2): โครงเดียวกับ _ask() ในหน้า IngredientsScreen
-    //   แต่เรียก widget.ai.analyzePhoto(bytes, mimeType: _mimeType)
-    ScaffoldMessenger.of(context).showSnackBar(
-      const SnackBar(content: Text('ยังไม่ได้เขียน _analyze() — ทำตามคู่มือ Step 7')),
-    );
+    setState(() {
+      _loading = true;
+      _error = null;
+      _result = null;
+    });
+    try {
+      final result = await widget.ai.analyzePhoto(bytes, mimeType: _mimeType);
+      if (!mounted) return;
+      setState(() => _result = result);
+    } on AiException catch (e) {
+      if (!mounted) return;
+      setState(() => _error = e.message);
+    } catch (e) {
+      if (!mounted) return;
+      setState(() => _error = 'อ่านคำตอบของ AI ไม่ได้: $e');
+    } finally {
+      if (mounted) setState(() => _loading = false);
+    }
   }
```

ภาคผนวก ค: Interactions API (ของใหม่จาก Google) ต่างจากที่เราใช้อย่างไร

ปลายปี 2026 Google เพิ่ม API ตัวใหม่ชื่อ **Interactions** (`POST /v1beta/interactions`) ที่เขียนสั้นกว่าและจำบทสนทนาฝั่งเซิร์ฟเวอร์ได้ ส่วน `generateContent` ที่เราใช้ Google ระบุว่ายัง *fully supported* และเป็นแบบที่เอกสาร/ตัวอย่างบนอินเทอร์เน็ตส่วนใหญ่ใช้ จึงเหมาะกับการเรียนรู้โครงสร้างพื้นฐานมากกว่า ถ้าอยากลองของใหม่ ลองเทียบดู:

```
POST https://generativelanguage.googleapis.com/v1beta/interactions
x-goog-api-key: <key>
{
  "model": "gemini-3.8-flash",
  "input": [
    { "type": "text",  "text": "ดูรูปนี้แล้วบอกว่ามีอะไรบ้าง" },
    { "type": "image", "data": "<base64>", "mime_type": "image/jpeg" }
  ],
  "system_instruction": "คุณคือเชฟ...",
  "response_format": { "type": "text", "mime_type": "application/json", "schema": { ... }
}
}
```

โจทย์เสริม: เขียน `InteractionsClient` ที่มี `generate()` หน้าตาเหมือน GeminiClient แต่ยิง endpoint นี้ แล้วสลับใช้ใน AppSettings — ถ้าออกแบบชั้น AiService ไว้ดี หน้าจอไม่ต้องแก้แม้แต่บรรทัดเดียว